



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Informática

Mención en Computación

Análisis del rendimiento y consumo de modelos de IA para detección de intrusiones

Performance and consumption analysis of AI models for intrusion detection

Realizado por

Plácido Velasco Muñoz

Tutorizado por

Dr. Rodrigo Román Castro

Co-tutorizado por

Dr. Francisco José Jaime Rodríguez

Departamento

Departamento de Lenguajes y Ciencias de la Computación

Málaga, April 21, 2026



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

GRADO EN INGENIERÍA INFORMÁTICA

Mención en Computación

**Análisis del rendimiento y consumo de modelos de
IA para detección de intrusiones**

**Performance and consumption analysis of AI
models for intrusion detection**

Realizado por

Plácido Velasco Muñoz

Tutorizado por

Dr. Rodrigo Román Castro

Co-tutorizado por

Dr. Francisco José Jaime Rodríguez

Departamento

Departamento de Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, APRIL 21, 2026

Resumen

Palabras clave:

Abstract

Keywords:

Agradecimientos

Resumen	1
Abstract	3
Agradecimientos	5
0.1 Investigación Previa	11
0.1.1 Machine Learning	11
0.1.2 Deep Learning	17
0.2 Obtención y Análisis de Datos	22
0.2.1 CIC-IDS-2017 DataSet	22
0.2.2 UNSW-NB15 DataSet	24
0.2.3 ToN_IoT DataSet	25
0.2.4 IoT-23 DataSet	27
0.2.5 Relación entre los datasets	29
0.2.6 Justificación del enfoque en la fase de inferencia	29
0.2.7 Optimización de hiperparámetros estructurales	30
0.3 Optuna	32
0.4 Resultados	33
0.4.1 Entorno de pruebas	33
0.4.2 Modelos entrenados con UNSW-NB15	33
0.4.3 Modelos entrenados con IOT-23	47

1	Esquema representativo del funcionamiento de Random Forest.	12
2	Esquema representativo del funcionamiento de XGBoost.	14
3	Esquema representativo del funcionamiento de LightGBM.	15
4	Esquema representativo del funcionamiento de CatBoost.	16
5	Ejemplo de separación óptima de clases mediante SVM y margen máximo.	17
6	Proceso de retropropagación en una red neuronal MLP.	19
7	Esquema representativo de una unidad LSTM con sus compuertas internas.	20
8	Esquema representativo de una arquitectura CNN 1D aplicada a datos tabulares. . . .	21
9	Logo del Canadian Institute for Cybersecurity, entidad responsable de la publicación del dataset CIC-IDS-2017.	24
10	Logo de la University of New South Wales, entidad responsable de la publicación del dataset UNSW-NB15.	25
11	Logo del Stratosphere Laboratory de la Czech Technical University, entidad responsable de la publicación del dataset IoT-23.	28
12	Visualización de la optimización con Optuna	33
13	Frontera de Pareto - Random Forest	34
14	Frontera de Pareto - XGBoost	35
15	Frontera de Pareto - LightGBM	36
16	Frontera de Pareto - CatBoost	37
17	Frontera de Pareto - SVM	39
18	Frontera de Pareto - MLP	40
19	Frontera de Pareto - LSTM	41
20	Frontera de Pareto - CNN	42
21	Curva ROC - Modelos	45
22	MC - Random Forest	45
23	MC - XGBoost	45
24	MC - LightGBM	46
25	MC - CatBoost	46
26	MC - SVM	46
27	MC - MLP	46
28	MC - LSTM	46
29	MC - CNN	46
30	Frontera de Pareto - Random Forest (IoT-23)	47
31	Frontera de Pareto - XGBoost (IoT-23)	48
32	Frontera de Pareto - LightGBM (IoT-23)	49

33	Frontera de Pareto - CatBoost (IOT-23)	50
34	Frontera de Pareto - SVM (IOT-23)	51
35	Frontera de Pareto - MLP (IOT-23)	52

1	Principales hiperparámetros de Random Forest y su significado.	12
2	Principales hiperparámetros de XGBoost y su significado.	13
3	Principales hiperparámetros de LightGBM y su significado.	14
4	Principales hiperparámetros de CatBoost y su significado.	15
5	Principales hiperparámetros de SVM y su significado.	17
6	Principales hiperparámetros de MLP y su significado.	18
7	Principales hiperparámetros de LSTM y su significado.	20
8	Principales hiperparámetros de CNN y su significado.	21
9	Distribución de clases y tipos de ataque en el dataset CIC-IDS-2017	23
10	Distribución de clases y tipos de ataque en el dataset UNSW-NB15	25
11	Distribución de clases y tipos de ataque en el dataset ToN_IoT	27
12	Distribución de clases y tipos de tráfico en el dataset IoT-23	28
13	Correspondencia principal entre características de CIC-IDS-2017 y UNSW-NB15 . .	29
14	Correspondencia principal entre características de ToN_IoT e IoT-23	29
15	Especificaciones del entorno experimental	33
16	Comparativa final de modelos Random Forest en el set de test	34
17	Comparativa final de modelos XGBoost en el set de test	36
18	Comparativa final de modelos LightGBM en el set de test	37
19	Comparativa final de modelos CatBoost en el set de test	38
20	Comparativa final de modelos SVM en el set de test	39
21	Comparativa final de modelos MLP en el set de test	40
22	Comparativa final de modelos LSTM en el set de test	41
23	Comparativa final de modelos CNN en el set de test	42
24	Definición técnica de métricas de monitoreo de recursos	43
25	Definición de métricas de evaluación de rendimiento	44
26	Tabla global de rendimiento de hardware	44
27	Comparativa final de modelos Random Forest en el set de test (IoT-23)	48
28	Comparativa final de modelos XGBoost en el set de test (IoT-23)	49
29	Comparativa final de modelos LightGBM en el set de test (IoT-23)	50
30	Comparativa final de modelos CatBoost en el set de test (IoT-23)	51
31	Comparativa final de modelos SVM en el set de test (IoT-23)	52
32	Comparativa final de modelos MLP en el set de test (IoT-23)	53

0.1 Investigación Previa

0.1.1 Machine Learning

El *Machine Learning* (aprendizaje automático) es una rama de la inteligencia artificial que permite a los sistemas aprender y mejorar automáticamente a partir de la experiencia, sin ser programados explícitamente para cada tarea. Utiliza algoritmos que identifican patrones en grandes volúmenes de datos y toman decisiones o realizan predicciones basadas en esos patrones.

En el contexto de los Sistemas de Detección de Intrusos (IDS), el *Machine Learning* se emplea para analizar el tráfico de red y detectar comportamientos anómalos o maliciosos de manera más eficiente y precisa que los métodos tradicionales basados en firmas. Además, la relación con la *Green AI* surge porque el uso de técnicas de aprendizaje automático más eficientes puede reducir el consumo energético, promoviendo soluciones más sostenibles.

0.1.1.1 Random Forest (RF)

Random Forest es un modelo de aprendizaje supervisado que combina múltiples árboles de decisión para obtener una predicción más robusta que la de un único árbol. En lugar de entrenar todos los árboles con exactamente los mismos datos, cada uno se construye a partir de una muestra diferente del conjunto de entrenamiento, generada mediante *bootstrap sampling*. Además, en cada división se selecciona un subconjunto aleatorio de características. Esta combinación de aleatoriedad en los datos y en las variables es clave para que el modelo generalice mejor.

A diferencia de los métodos de *boosting*, como XGBoost, los árboles que componen un Random Forest no se entrenan de manera secuencial ni dependen unos de otros. Cada árbol aprende por su cuenta y aporta su propia "opinión", que posteriormente se integra con la del resto, normalmente mediante votación mayoritaria en clasificación. Esta independencia reduce la varianza del modelo y lo hace menos sensible al ruido o a pequeñas variaciones en los datos, algo especialmente útil en escenarios como la detección de intrusiones.

Desde el punto de vista de la inferencia, Random Forest presenta una latencia muy reducida, ya que la predicción consiste únicamente en recorrer un número limitado de árboles poco profundos y agregar sus decisiones. Además, al no requerir cálculos iterativos complejos ni funciones de activación costosas, su consumo computacional durante la fase de operación es bajo, lo que lo convierte en una opción adecuada para despliegues en entornos con recursos limitados.

Si se analiza desde la perspectiva de la *Green AI*, Random Forest ofrece un compromiso razonable entre capacidad predictiva y coste computacional. Aunque el entrenamiento implica construir varios árboles y puede resultar más pesado que el de un modelo simple, este proceso se realiza una sola vez. En contraste, la fase de inferencia es altamente eficiente, lo que resulta especialmente relevante en sistemas de detección de intrusiones.

Tabla 1. Principales hiperparámetros de Random Forest y su significado.

Hiperparámetro	Significado
<code>n_estimators</code>	Número de árboles en el bosque
<code>max_depth</code>	Profundidad máxima de cada árbol
<code>min_samples_split</code>	Mínimo de muestras para dividir un nodo
<code>min_samples_leaf</code>	Mínimo de muestras en una hoja
<code>max_features</code>	Número de características consideradas en cada división
<code>bootstrap</code>	Si se usan muestras con reemplazo para entrenar cada árbol
<code>criterion</code>	Función para medir la calidad de una división (por ejemplo, <i>gini</i> o <i>entropy</i>)

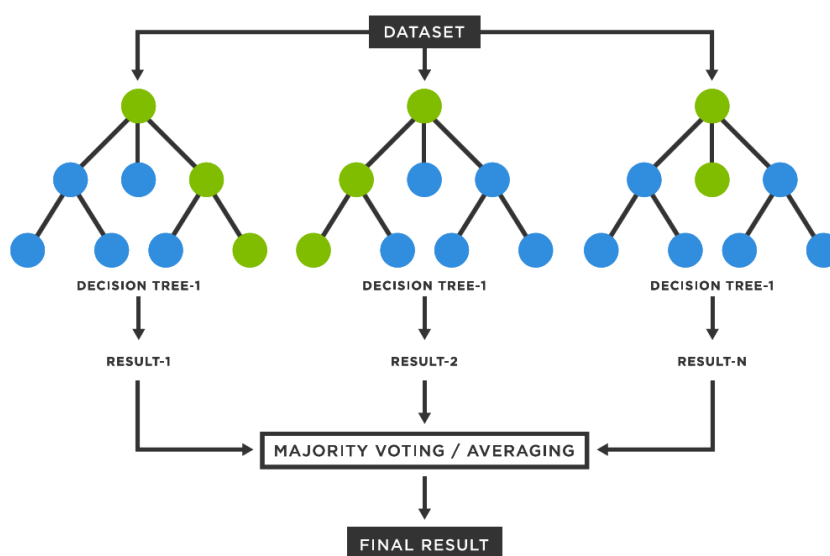


Figura 1. Esquema representativo del funcionamiento de Random Forest.

0.1.1.2 Extreme Gradient Boosting (XGBoost)

XGBoost (*Extreme Gradient Boosting*) es un algoritmo de aprendizaje supervisado basado en árboles de decisión que implementa de forma altamente optimizada el principio de *gradient boosting*. A diferencia de otros métodos basados en ensamblados de árboles, XGBoost construye el modelo de manera secuencial, añadiendo nuevos árboles que se centran en corregir los errores cometidos por los árboles previamente entrenados. Este proceso se basa en la optimización iterativa de una función objetivo que combina el error de clasificación con términos explícitos de regularización, lo que permite controlar la complejidad del modelo y reducir el sobreajuste.

En cada iteración, XGBoost utiliza información de primer (dirección) y segundo (qué tan pronunciado es el plano) orden del gradiente de la función de pérdida para determinar la estructura óptima de los árboles, lo que le permite capturar relaciones no lineales complejas e interacciones entre características de forma eficiente. Estas propiedades lo convierten en una opción especialmente adecuada para

la detección de intrusiones en conjuntos de datos tabulares, donde los patrones maliciosos suelen manifestarse a través de combinaciones no triviales de múltiples variables.

En XGBoost, la predicción final del modelo no corresponde a la salida de un único árbol de decisión, sino que se obtiene como la combinación aditiva de las salidas de todos los árboles entrenados. Cada árbol aporta una contribución parcial a la predicción, y el resultado final se calcula sumando dichas contribuciones. El proceso de entrenamiento es secuencial: cada nuevo árbol se entrena para corregir los errores residuales cometidos por el conjunto de árboles previamente construidos. Como consecuencia, el último árbol no contiene una decisión completa por sí mismo, sino que únicamente modela aquellos patrones que aún no han sido correctamente capturados por los árboles anteriores, actuando como un ajuste fino del modelo global. De este modo, XGBoost se centra en la reducción del sesgo, lo que suele traducirse en un mayor rendimiento predictivo cuando se dispone de un número limitado de características relevantes.

En el marco de este trabajo, XGBoost se considera una alternativa sólida no solo por su capacidad de detección, sino también por su relación entre rendimiento y coste computacional. Es cierto que el entrenamiento puede resultar exigente, especialmente si se ajustan múltiples hiperparámetros. Sin embargo, este proceso se realiza de manera puntual. Una vez entrenado, el modelo ofrece tiempos de inferencia bajos, ya que la predicción se limita a recorrer un conjunto de árboles relativamente pequeños y sumar sus salidas. Esta característica facilita su uso en escenarios IoT. En consecuencia, XGBoost encaja razonablemente bien dentro de los principios de *Green AI*, al permitir un buen nivel de detección sin penalizar en exceso el consumo durante la operación continua del IDS.

Tabla 2. Principales hiperparámetros de XGBoost y su significado.

Hiperparámetro	Significado
n_estimators	Número de árboles a construir
max_depth	Profundidad máxima de cada árbol
learning_rate	Tasa de aprendizaje (contribución de cada árbol)
subsample	Proporción de muestras usadas en cada iteración
colsample_bytree	Proporción de características usadas por árbol
gamma	Mínima reducción de pérdida para dividir un nodo
reg_alpha	Término de regularización L1
reg_lambda	Término de regularización L2
objective	Función objetivo a optimizar (por ejemplo, <i>binary:logistic</i>)

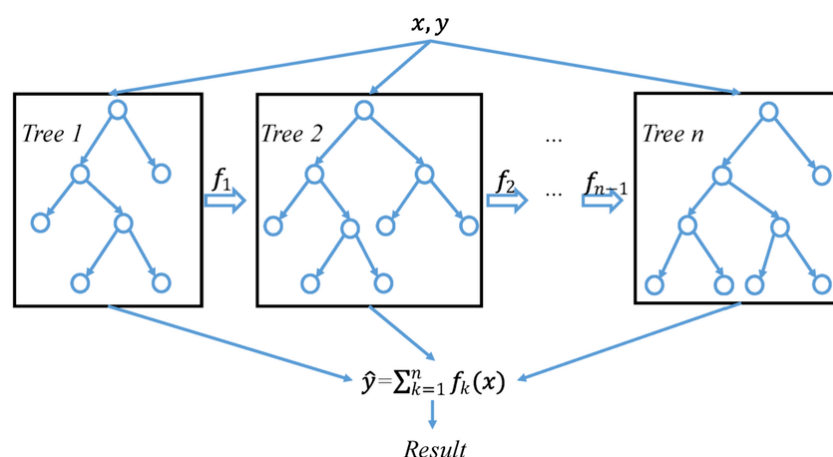


Figura 2. Esquema representativo del funcionamiento de XGBoost.

0.1.1.3 Light Gradient Boosting Machine (LightGBM)

Light Gradient Boosting Machine (LightGBM) es un algoritmo de aprendizaje supervisado basado en árboles de decisión que implementa una variante optimizada del *gradient boosting*. A diferencia de otros métodos de boosting, LightGBM utiliza un enfoque de crecimiento de árboles basado en hojas, lo que significa que en cada iteración se expande la hoja con el mayor error en lugar de expandir todos los nodos a un nivel determinado. Esta estrategia permite construir árboles más profundos y complejos sin aumentar significativamente el número de nodos, lo que mejora la capacidad de modelado y reduce el tiempo de entrenamiento.

Respecto a la inferencia, LightGBM ofrece tiempos de predicción muy bajos, ya que la estructura de los árboles es más eficiente y se optimiza para reducir el número de nodos necesarios para realizar una predicción. Además, su implementación está diseñada para aprovechar al máximo los recursos computacionales disponibles, lo que contribuye a minimizar el consumo energético durante la fase de inferencia. En el contexto de la *Green AI*, LightGBM representa una opción atractiva, ya que combina eficiencia computacional con capacidad de modelado. LightGBM es particularmente ventajoso en escenarios donde se requiere minimizar la latencia de inferencia y el consumo energético, características fundamentales para sistemas IDS desplegados en entornos con recursos limitados o dispositivos IoT.

Tabla 3. Principales hiperparámetros de LightGBM y su significado.

Hiperparámetro	Significado
<code>n_estimators</code>	Número de árboles a construir
<code>num_leaves</code>	Número máximo de hojas en cada árbol
<code>max_depth</code>	Profundidad máxima de cada árbol
<code>learning_rate</code>	Tasa de aprendizaje (contribución de cada árbol)
<code>n_jobs</code>	Número de threads para el entrenamiento paralelo
<code>random_state</code>	Semilla para reproducibilidad
<code>verbosity</code>	Nivel de detalle en los mensajes de salida

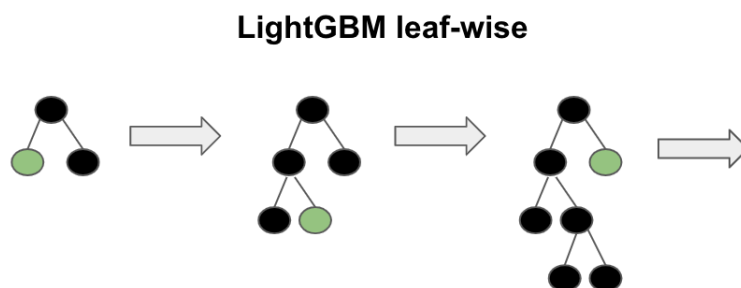


Figura 3. Esquema representativo del funcionamiento de LightGBM.

0.1.1.4 CatBoost

CatBoost es un algoritmo de aprendizaje supervisado basado en árboles de decisión que implementa una variante optimizada del *gradient boosting*, desarrollada por Yandex.

Una de las principales características de CatBoost es su capacidad para manejar de manera eficiente variables categóricas sin necesidad de realizar una codificación previa, lo que simplifica el proceso de preparación de datos y mejora el rendimiento del modelo.

Otro punto clave es que a diferencia de otros modelos como XGBoost o LightGBM, CatBoost crea árboles simétricos, lo que nos da una latencia de inferencia muy baja, ya que cada árbol tiene la misma profundidad y se puede recorrer de manera más eficiente. Además, su implementación está optimizada para aprovechar al máximo los recursos computacionales disponibles, lo que contribuye a minimizar el consumo energético durante la fase de inferencia.

CatBoost es particularmente ventajoso en escenarios donde se requiere minimizar la latencia de inferencia y el consumo energético, características fundamentales para sistemas IDS desplegados en entornos con recursos limitados o dispositivos IoT.

Tabla 4. Principales hiperparámetros de CatBoost y su significado.

Hiperparámetro	Significado
<code>iterations</code>	Número de árboles a construir
<code>depth</code>	Profundidad máxima de cada árbol
<code>learning_rate</code>	Tasa de aprendizaje (contribución de cada árbol)
<code>cat_features</code>	Índices de características categóricas
<code>loss_function</code>	Función de pérdida a optimizar
<code>random_seed</code>	Semilla para reproducibilidad

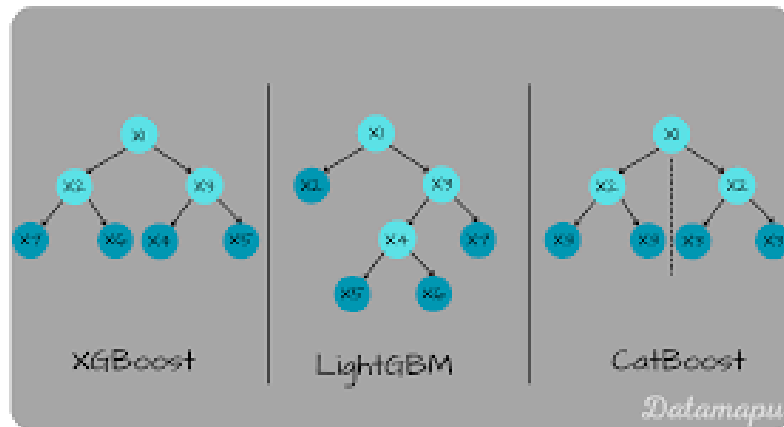


Figura 4. Esquema representativo del funcionamiento de CatBoost.

0.1.1.5 Support Vector Machine (SVM)

Support Vector Machines (SVM) es un algoritmo de aprendizaje supervisado cuyo objetivo principal es encontrar una frontera de decisión que separe de forma óptima las distintas clases en el espacio de características. En su formulación básica, SVM busca el hiperplano que maximiza el margen entre las muestras de distintas clases, utilizando únicamente un subconjunto de los datos denominado *vectores soporte*. Esta propiedad permite construir modelos robustos frente al ruido y con buena capacidad de generalización.

Una de las principales fortalezas de las SVM es el uso de *kernels*, que permiten proyectar los datos originales a espacios de mayor dimensionalidad sin realizar explícitamente dicha transformación. Gracias a este mecanismo, SVM puede modelar fronteras de decisión no lineales incluso cuando los datos no son separables linealmente en el espacio original. Entre los kernels más utilizados se encuentran:

- **Kernel lineal:** Define una frontera de decisión lineal en el espacio de características original. Es el kernel más simple y el más eficiente desde el punto de vista computacional, lo que lo hace especialmente adecuado para datasets de gran tamaño y para escenarios donde se prioriza la eficiencia.
- **Kernel polinómico:** Permite modelar relaciones no lineales mediante funciones polinómicas de distinto grado. Aunque puede capturar interacciones más complejas entre las características, su coste computacional aumenta rápidamente con el grado del polinomio y el tamaño del dataset.
- **Kernel radial (RBF):** Proyecta los datos a un espacio de dimensión infinita, permitiendo separar patrones altamente no lineales. Si bien suele ofrecer un alto rendimiento predictivo, su entrenamiento e inferencia son significativamente más costosos, especialmente en conjuntos de datos de gran tamaño.

En el contexto de este trabajo, el uso de SVM se plantea principalmente mediante un **kernel lineal**, ya que ofrece un compromiso adecuado entre rendimiento y eficiencia computacional. Los kernels no lineales, como el RBF, resultan poco escalables para los volúmenes de datos considerados y presentan un coste energético elevado, lo que los hace menos compatibles con los principios de *Green AI*.

Tabla 5. Principales hiperparámetros de SVM y su significado.

Hiperparámetro	Significado
C	Parámetro de regularización que controla el equilibrio entre margen y error de clasificación
kernel	Tipo de función kernel utilizada (por ejemplo, <i>linear</i> , <i>poly</i> , <i>rbf</i>)
degree	Grado del polinomio (solo para kernel polinómico)
gamma	Coefficiente para kernels <i>rbf</i> , <i>poly</i> y <i>sigmoid</i>
coef0	Término independiente en kernels <i>poly</i> y <i>sigmoid</i>

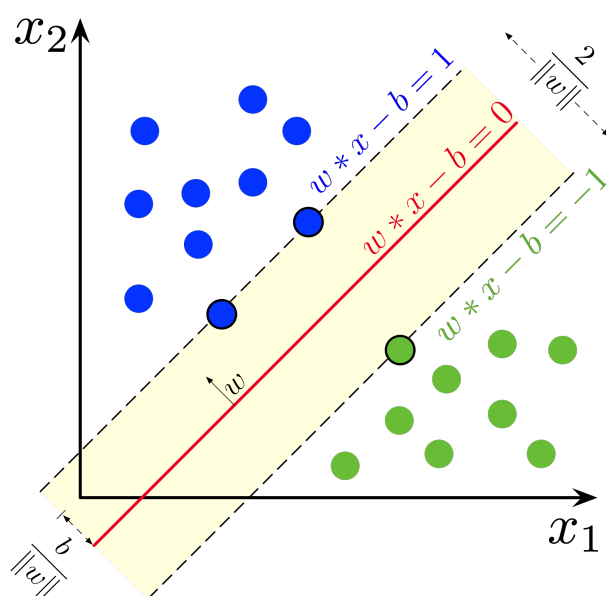


Figura 5. Ejemplo de separación óptima de clases mediante SVM y margen máximo.

0.1.2 Deep Learning

El *Deep Learning* (aprendizaje profundo) es una subárea del aprendizaje automático que se basa en redes neuronales artificiales con múltiples capas (redes neuronales profundas). Estas redes son capaces de aprender representaciones jerárquicas de los datos, lo que les permite capturar patrones complejos y realizar tareas como clasificación, reconocimiento de imágenes y procesamiento del lenguaje natural.

En el ámbito de los IDS, el *Deep Learning* se utiliza para mejorar la detección de intrusiones mediante la capacidad de aprender características complejas del tráfico de red. Sin embargo, es importante considerar que los modelos de aprendizaje profundo suelen requerir una gran cantidad de datos y

recursos computacionales, lo que puede aumentar el consumo energético. Por lo tanto, es crucial buscar enfoques que optimicen el rendimiento sin comprometer la eficiencia energética.

0.1.2.1 Multilayer Perceptron (MLP)

El **Multi-Layer Perceptron** (MLP) es una red neuronal artificial de tipo *feed-forward* compuesta por una capa de entrada, una o varias capas ocultas y una capa de salida. Cada capa está formada por neuronas completamente conectadas, donde cada neurona realiza una combinación lineal de sus entradas seguida de una función de activación no lineal. Este tipo de arquitectura permite aproximar funciones no lineales complejas y es uno de los modelos más utilizados como punto de partida en problemas de aprendizaje profundo.

En el contexto de datasets tabulares, como los empleados en este trabajo, el MLP actúa como un modelo de referencia dentro del aprendizaje profundo, permitiendo evaluar si la introducción de capas no lineales aporta mejoras significativas frente a modelos clásicos como Random Forest o XGBoost. A diferencia de estos últimos, el rendimiento de un MLP depende en gran medida del diseño de la arquitectura y de la correcta selección de hiperparámetros.

Desde el punto de vista computacional, su coste está directamente relacionado con el número de capas ocultas y el número de neuronas por capa. Arquitecturas profundas o excesivamente anchas incrementan de forma notable el tiempo de entrenamiento y el consumo de memoria, sin garantizar necesariamente mejoras en el rendimiento para datos tabulares. Por este motivo, en este trabajo se utilizan arquitecturas MLP compactas, con un número reducido de capas y neuronas, priorizando un equilibrio adecuado entre capacidad de modelado y eficiencia computacional.

MLP representa una alternativa intermedia entre los modelos de aprendizaje automático clásico y redes neuronales más complejas como LSTM o CNN. Aunque su entrenamiento es más costoso que el de modelos basados en árboles, la fase de inferencia sigue siendo relativamente eficiente cuando se emplean arquitecturas ligeras, lo que permite analizar el compromiso entre rendimiento y coste computacional en escenarios de detección de intrusiones.

Tabla 6. Principales hiperparámetros de MLP y su significado.

Hiperparámetro	Significado
<code>hidden_layer_sizes</code>	Número y tamaño de las capas ocultas
<code>activation</code>	Función de activación utilizada en las neuronas (por ejemplo, <i>relu</i> , <i>tanh</i>)
<code>solver</code>	Algoritmo de optimización para el entrenamiento (por ejemplo, <i>adam</i> , <i>sgd</i>)
<code>alpha</code>	Parámetro de regularización L2
<code>learning_rate_init</code>	Tasa de aprendizaje inicial
<code>batch_size</code>	Tamaño del lote de muestras para cada actualización de los pesos
<code>max_iter</code>	Número máximo de iteraciones de entrenamiento
<code>early_stopping</code>	Si se detiene el entrenamiento anticipadamente al no mejorar el rendimiento

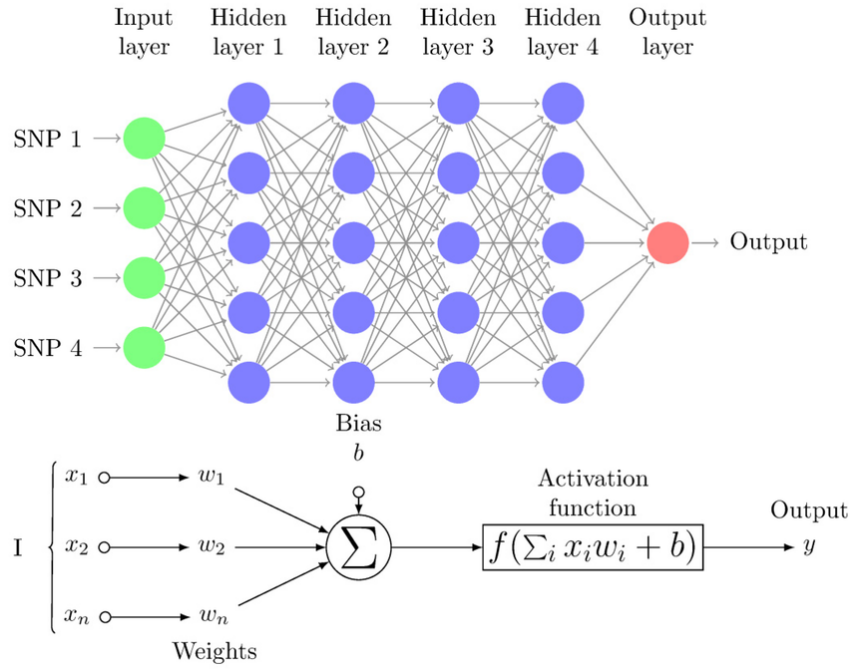


Figura 6. Proceso de retropropagación en una red neuronal MLP.

0.1.2.2 Long Short-Term Memory (LSTM)

Las redes **Long Short-Term Memory** (LSTM) son un tipo de red neuronal recurrente diseñada para modelar dependencias temporales en secuencias de datos. A diferencia de las redes neuronales recurrentes tradicionales, las LSTM incorporan un conjunto de compuertas internas (entrada, olvido y salida) que regulan el flujo de información a lo largo del tiempo, permitiendo conservar o descartar información relevante durante secuencias largas y mitigando el problema del desvanecimiento del gradiente (común en redes recurrentes cuya consecuencia es la dificultad de recordar información lejana).

En el ámbito de la detección de intrusiones, las LSTM resultan especialmente interesantes cuando el comportamiento malicioso no se manifiesta en observaciones aisladas, sino en patrones temporales que evolucionan a lo largo del tiempo. Aunque los datasets empleados en este trabajo se describen mediante características agregadas a nivel de flujo, el uso de LSTM permite analizar si la introducción de mecanismos de memoria aporta ventajas frente a modelos puramente estáticos, capturando posibles dependencias internas entre características o secuencias de flujos consecutivos.

Desde el punto de vista computacional, las LSTM presentan un coste superior al de modelos *feed-forward* como el MLP, ya que cada unidad recurrente realiza múltiples operaciones internas en cada paso temporal. El número de unidades LSTM y la longitud de las secuencias influyen directamente en el tiempo de entrenamiento y en el consumo de memoria. Por este motivo, en este trabajo se emplean arquitecturas LSTM ligeras, con un número reducido de unidades, evitando configuraciones profundas que resultarían inviables para escenarios con recursos limitados.

En términos de *Green AI*, las LSTM representan un compromiso entre capacidad de modelado y eficiencia. Si bien su entrenamiento e inferencia son más costosos que los de modelos clásicos y MLP sencillos, el uso de configuraciones compactas permite evaluar de forma controlada el impacto de la modelización temporal en el rendimiento del sistema IDS. De este modo, las LSTM se incluyen como un modelo de referencia para analizar el beneficio potencial de incorporar memoria temporal frente al incremento del coste computacional asociado.

Tabla 7. Principales hiperparámetros de LSTM y su significado.

Hiperparámetro	Significado
<code>units</code>	Número de unidades LSTM en la capa recurrente
<code>return_sequences</code>	Si la capa devuelve la secuencia completa o solo el último estado
<code>dropout</code>	Tasa de abandono para regularización durante el entrenamiento
<code>recurrent_dropout</code>	Tasa de abandono aplicada a las conexiones recurrentes
<code>activation</code>	Función de activación utilizada en las unidades LSTM (por ejemplo, <i>tanh</i>)
<code>recurrent_activation</code>	Función de activación para las compuertas internas (por ejemplo, <i>sigmoid</i>)
<code>learning_rate_init</code>	Tasa de aprendizaje inicial para el optimizador
<code>batch_size</code>	Tamaño del lote de muestras para cada actualización de los pesos
<code>max_iter</code>	Número máximo de iteraciones de entrenamiento

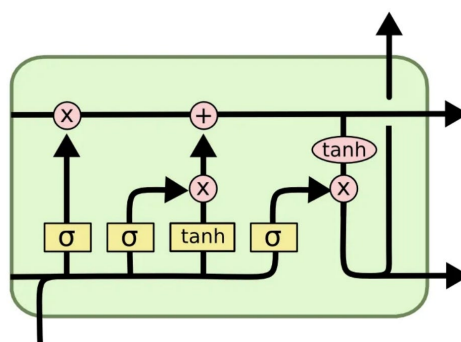


Figura 7. Esquema representativo de una unidad LSTM con sus compuertas internas.

0.1.2.3 Convolutional Neural Networks (CNN)

Las **Convolutional Neural Networks** (CNN) son un tipo de red neuronal profunda diseñadas originalmente para el procesamiento de datos con estructura espacial, como imágenes o señales. Su principal característica es el uso de capas convolucionales, que aplican filtros locales sobre los datos de entrada para extraer patrones relevantes de forma jerárquica, reduciendo al mismo tiempo el número de parámetros respecto a redes completamente conectadas.

En el contexto de este trabajo, las CNN se emplean en su variante *1D*, donde las convoluciones se aplican sobre el vector de características de cada flujo de red, tratado como una señal unidimensional.

Este enfoque permite capturar relaciones locales entre características adyacentes, modelando interacciones de corto alcance entre variables sin necesidad de introducir arquitecturas excesivamente complejas. De este modo, la CNN 1D actúa como una alternativa intermedia entre el MLP y las redes recurrentes, explorando si la extracción local de patrones aporta mejoras en la detección de intrusiones.

Desde el punto de vista computacional, el coste de una CNN depende principalmente del número de filtros, el tamaño del kernel y la profundidad de la red. Aunque las CNN suelen ser más eficientes que otras arquitecturas profundas al compartir pesos, un número elevado de filtros o capas puede incrementar significativamente el tiempo de entrenamiento y la latencia de inferencia. Por esta razón, en este trabajo se utilizan arquitecturas CNN poco profundas y con un número reducido de filtros, priorizando configuraciones compactas y eficientes.

Tabla 8. Principales hiperparámetros de CNN y su significado.

Hiperparámetro	Significado
<code>filters</code>	Número de filtros en la capa convolucional
<code>kernel_size</code>	Tamaño del kernel de convolución (por ejemplo, 3)
<code>strides</code>	Paso de la convolución sobre la entrada
<code>padding</code>	Tipo de relleno aplicado a los bordes (por ejemplo, <i>same</i> , <i>valid</i>)
<code>activation</code>	Función de activación utilizada en las capas convolucionales (por ejemplo, <i>relu</i>)
<code>learning_rate_init</code>	Tasa de aprendizaje inicial para el optimizador
<code>batch_size</code>	Tamaño del lote de muestras para cada actualización de los pesos
<code>max_iter</code>	Número máximo de iteraciones de entrenamiento

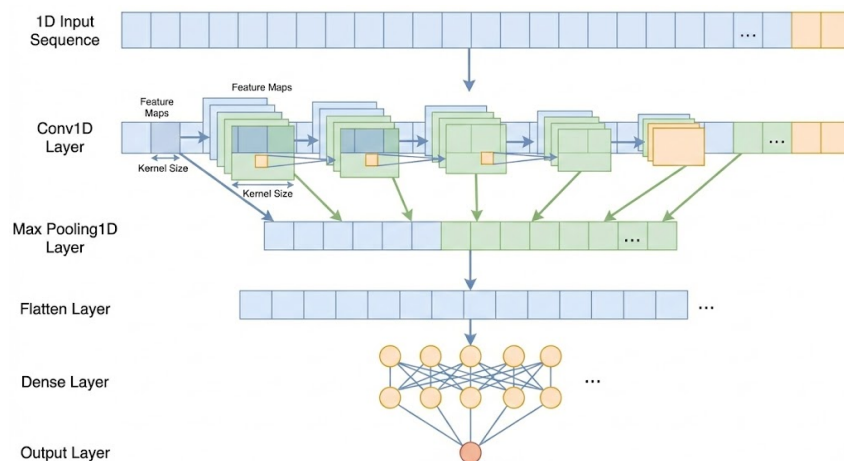


Figura 8. Esquema representativo de una arquitectura CNN 1D aplicada a datos tabulares.

En resumen, la selección de modelos de aprendizaje automático y profundo para la detección de intrusiones debe considerar no solo el rendimiento predictivo, sino también el coste computacional asociado. Modelos como Random Forest y XGBoost ofrecen un buen equilibrio entre capacidad de modelado y eficiencia, mientras que las redes neuronales, aunque potencialmente más potentes, requieren un diseño cuidadoso para evitar un consumo energético excesivo.

0.2 Obtención y Análisis de Datos

Para cumplir con los objetivos de este Trabajo Fin de Grado, resulta imprescindible realizar una selección cuidadosa de los conjuntos de datos empleados en la fase experimental. En el contexto de los sistemas de detección de intrusiones basados en inteligencia artificial, los datasets no solo deben ser representativos de una amplia diversidad de ataques y comportamientos de red, sino que también deben permitir una **evaluación realista y rigurosa** de los modelos entrenados.

En particular, dado que este trabajo no se centra únicamente en la capacidad de detección, sino también en el rendimiento y el consumo de recursos computacionales, los conjuntos de datos seleccionados deben posibilitar el análisis de aspectos como la latencia de inferencia, el uso de CPU y memoria, y la escalabilidad de los modelos en distintos escenarios. El uso de datasets excesivamente simplificados o poco representativos podría conducir a conclusiones poco realistas, mientras que el empleo de conjuntos de datos desproporcionadamente grandes o complejos podría dificultar la reproducibilidad y el análisis experimental detallado.

Por este motivo, en este proyecto se han seleccionado datasets ampliamente utilizados en la literatura científica, que cubren diferentes entornos y tipologías de ataque, y que ofrecen un equilibrio adecuado entre realismo, diversidad y viabilidad experimental. Esta elección permite evaluar de forma comparativa el comportamiento de distintos modelos de aprendizaje automático y profundo.

0.2.1 CIC-IDS-2017 DataSet

0.2.1.1 Motivación para el uso del dataset

El conjunto de datos CIC-IDS-2017 ha sido seleccionado como uno de los datasets principales para la evaluación experimental de los modelos de detección de intrusiones propuestos en este trabajo. Publicado por el *Canadian Institute for Cybersecurity (CIC)*, este dataset se ha consolidado como una referencia en la literatura reciente sobre sistemas de detección de intrusiones basados en técnicas de aprendizaje automático y aprendizaje profundo.

La elección de CIC-IDS-2017 se justifica, en primer lugar, por el elevado grado de realismo de los datos, ya que el tráfico fue generado en un entorno de red que simula el comportamiento de una infraestructura corporativa real. El dataset incluye tráfico benigno y malicioso capturado a lo largo de varios días consecutivos, incorporando una amplia variedad de ataques representativos de escenarios reales, como ataques de denegación de servicio (DoS y DDoS), escaneos de puertos, ataques de fuerza bruta, ataques web y tráfico asociado a botnets.

Asimismo, el uso extensivo de CIC-IDS-2017 en trabajos previos permite comparar de forma directa los resultados obtenidos en este estudio con los de la literatura existente, reforzando la validez experimental de las conclusiones. A pesar de presentar limitaciones conocidas, como el desbalanceo entre clases o la presencia de ataques con muy baja frecuencia, estas características reflejan problemas

habituales en entornos reales de ciberseguridad y constituyen un reto adicional para la evaluación de los modelos.

0.2.1.2 Estructura del dataset

El dataset CIC-IDS-2017 está organizado en varios archivos CSV, cada uno de los cuales corresponde a un día concreto de actividad en la red y a un conjunto específico de escenarios de ataque. Cada registro del dataset representa un flujo de red, descrito mediante un conjunto de características extraídas a nivel de flujo, lo que facilita su uso con modelos de aprendizaje automático aplicados a datos tabulares.

Entre las características incluidas se encuentran métricas relacionadas con la duración del flujo, el número y tamaño de los paquetes enviados y recibidos, estadísticas temporales, información sobre flags TCP y otros atributos derivados del comportamiento del tráfico. Además, el dataset incluye una etiqueta denominada `Label`, que identifica si el flujo corresponde a tráfico benigno o a un tipo concreto de ataque, permitiendo abordar tanto problemas de clasificación binaria como multiclase.

0.2.1.3 Distribución de ataques

La distribución de clases del dataset CIC-IDS-2017 se caracteriza por un fuerte desbalanceo entre el tráfico benigno y el tráfico malicioso, así como por una distribución heterogénea entre los distintos tipos de ataque. En la Tabla 9 se presenta el número total de flujos correspondiente a cada clase, considerando el conjunto completo del dataset.

Tabla 9. Distribución de clases y tipos de ataque en el dataset CIC-IDS-2017

Clase / Tipo de tráfico	Número de flujos
BENIGN	2 273 097
DoS Hulk	231 073
PortScan	158 930
DDoS	128 027
DoS GoldenEye	10 293
FTP-Patator	7 938
SSH-Patator	5 897
DoS slowloris	5 796
DoS Slowhttptest	5 499
Bot	1 966
Web Attack – Brute Force	1 507
Web Attack – XSS	652
Infiltration	36
Web Attack – SQL Injection	21
Heartbleed	11
Total	2 830 743

Como puede observarse, el tráfico benigno representa la mayor parte de los datos, mientras que los ataques se distribuyen de forma muy desigual entre las distintas categorías. Clases como *DoS Hulk*,

PortScan y *DDoS* concentran la mayor parte del tráfico malicioso, mientras que otros ataques aparecen de forma testimonial. Esta distribución refuerza la necesidad de aplicar estrategias específicas para el tratamiento del desbalanceo durante el proceso de entrenamiento y evaluación de los modelos de detección de intrusiones.



Figura 9. Logo del Canadian Institute for Cybersecurity, entidad responsable de la publicación del dataset CIC-IDS-2017.

0.2.2 UNSW-NB15 DataSet

0.2.2.1 Motivación para el uso del dataset

El conjunto de datos UNSW-NB15 se ha seleccionado como dataset complementario en este trabajo con el objetivo de reforzar la validez de los resultados obtenidos y evaluar el comportamiento de los modelos de detección de intrusiones en un entorno de datos distinto al representado por CIC-IDS-2017. Este dataset fue desarrollado por la *University of New South Wales (UNSW)* con el propósito de abordar algunas de las limitaciones detectadas en conjuntos de datos más antiguos y proporcionar un escenario más realista y actualizado para la evaluación de sistemas IDS.

La inclusión de UNSW-NB15 permite analizar la capacidad de generalización de los modelos, contrastando si las conclusiones obtenidas con CIC-IDS-2017 se mantienen cuando se trabaja con un dataset que presenta una distribución de datos, un conjunto de características y una tipología de ataques diferentes. De este modo, se reduce la dependencia de los resultados respecto a un único conjunto de datos y se refuerza la robustez experimental del estudio.

0.2.2.2 Estructura del dataset

El dataset UNSW-NB15 está compuesto por registros de tráfico de red generados mediante la combinación de tráfico real y tráfico sintético, con el fin de simular de forma controlada distintos escenarios de ataque en una red corporativa. El conjunto de datos se distribuye en dos archivos principales claramente diferenciados: uno destinado al entrenamiento de los modelos y otro reservado para su evaluación, lo que facilita la reproducibilidad experimental y minimiza el riesgo de *data leakage*.

Cada registro representa un flujo de red y se describe mediante un conjunto de características heterogéneas que incluyen métricas relacionadas con el volumen de tráfico, estadísticas temporales, in-

formación de protocolos y atributos derivados del comportamiento de la comunicación. En cuanto a las etiquetas, el dataset incorpora una etiqueta binaria que distingue entre tráfico benigno y tráfico malicioso, así como una etiqueta adicional que especifica el tipo de ataque, lo que permite abordar tanto problemas de clasificación binaria como multiclase.

0.2.2.3 Distribución de ataques

La distribución de clases del dataset UNSW-NB15 presenta un desbalanceo significativo entre el tráfico benigno y las distintas categorías de ataque, así como una variabilidad notable entre los diferentes tipos de tráfico malicioso. En la Tabla 10 se muestra el número total de flujos correspondiente a cada clase, considerando conjuntamente los conjuntos de entrenamiento y prueba.

Tabla 10. Distribución de clases y tipos de ataque en el dataset UNSW-NB15

Clase / Tipo de tráfico	Número de flujos
BENIGN	93 000
Generic	58 871
Exploits	44 525
Fuzzers	24 246
DoS	16 353
Reconnaissance	13 987
Analysis	2 677
Backdoor	2 329
Shellcode	1 511
Worms	174
Total	257 673

Como puede observarse, aunque el tráfico benigno constituye una parte relevante del dataset, una proporción significativa de los registros corresponde a tráfico malicioso, distribuido entre múltiples categorías de ataque. Destacan especialmente las clases *Generic*, *Exploits* y *Fuzzers*, mientras que otras, como *Worms* o *Shellcode*, presentan una frecuencia mucho menor. Esta distribución refleja un escenario realista y resulta adecuada para evaluar el comportamiento de los modelos tanto en términos de detección general de intrusiones como de clasificación específica de ataques.



Figura 10. Logo de la University of New South Wales, entidad responsable de la publicación del dataset UNSW-NB15.

0.2.3 ToN_IoT DataSet

0.2.3.1 Motivación para el uso del dataset

El conjunto de datos ToN_IoT (*Telemetry of Network IoT*) se ha incluido en este trabajo con el objetivo de evaluar el comportamiento de los modelos de detección de intrusiones en entornos IoT y *edge computing*, caracterizados por una elevada heterogeneidad, recursos computacionales limitados y requisitos estrictos de eficiencia. Este dataset fue desarrollado por la *University of New South Wales (UNSW)* con el propósito de cubrir las limitaciones de los conjuntos de datos tradicionales, que no representan adecuadamente el tráfico ni los patrones de ataque presentes en infraestructuras IoT reales.

De este modo, ToN_IoT complementa a los otros datasets empleados en el estudio, ampliando el alcance experimental hacia entornos más realistas desde el punto de vista del despliegue de soluciones de detección de intrusiones basadas en inteligencia artificial.

0.2.3.2 Estructura del dataset

El dataset ToN_IoT se distribuye en múltiples subconjuntos que incluyen tráfico de red, telemetría de dispositivos IoT y registros de sistemas operativos. No obstante, con el fin de mantener la comparabilidad con los otros conjuntos de datos utilizados y reducir la complejidad experimental, en este trabajo se ha seleccionado exclusivamente el **subconjunto de tráfico de red**.

En concreto, se ha empleado el archivo `Train_Test_Network_dataset.csv`, que proporciona los datos en formato CSV e incluye una partición predefinida de entrenamiento y prueba. Cada registro representa un flujo de red y se describe mediante un conjunto de características extraídas automáticamente, que abarcan métricas relacionadas con el volumen de tráfico, estadísticas temporales, información de protocolos y atributos derivados del comportamiento de la comunicación.

En cuanto al etiquetado, el dataset incorpora una etiqueta binaria que distingue entre tráfico benigno y tráfico malicioso, así como una etiqueta adicional que identifica el tipo de ataque. Esta estructura permite abordar tanto el problema de detección binaria de intrusiones como el análisis de los distintos tipos de tráfico malicioso presentes en entornos IoT.

0.2.3.3 Distribución de ataques

La distribución de clases del subconjunto de red de ToN_IoT presenta una proporción equilibrada entre tráfico benigno y varias categorías de ataques, lo que resulta especialmente interesante para evaluar el comportamiento de los modelos en un escenario IoT controlado. En la Tabla 11 se muestra el número total de flujos correspondiente a cada clase, considerando el conjunto completo de datos.

Como puede observarse, el dataset incluye múltiples tipos de ataques representativos de entornos IoT, tales como ataques de denegación de servicio, inyección, escaneo, ransomware y ataques de intermediario (*Man-in-the-Middle*). La distribución relativamente equilibrada entre varias categorías facilita el análisis comparativo del rendimiento de los modelos y permite evaluar su capacidad para detectar

diferentes patrones de comportamiento malicioso en escenarios con recursos limitados.

Tabla 11. Distribución de clases y tipos de ataque en el dataset *ToN_IoT*

Clase / Tipo de tráfico	Número de flujos
BENIGN	50 000
backdoor	20 000
ddos	20 000
dos	20 000
injection	20 000
password	20 000
ransomware	20 000
scanning	20 000
xss	20 000
mitm	1 043
Total	191 043

0.2.4 IoT-23 Data Set

0.2.4.1 Motivación para el uso del dataset

El conjunto de datos **IoT-23** se ha incorporado en este trabajo como dataset complementario orientado a entornos IoT reales, con el objetivo de analizar el comportamiento de los modelos de detección de intrusiones frente a tráfico real, alta variabilidad y limitaciones prácticas de procesamiento. IoT-23, desarrollado por el grupo *Stratosphere IPS* de la *Czech Technical University (CTU)*, es uno de los conjuntos de datos más representativos para el estudio de amenazas dirigidas específicamente a dispositivos IoT.

A diferencia de otros datasets generados en entornos controlados o sintéticos, IoT-23 se basa en capturas reales de tráfico de red procedentes de dispositivos IoT infectados con malware. Cada escenario refleja el comportamiento de un dispositivo comprometido durante periodos prolongados de tiempo, incluyendo actividad asociada a botnets, escaneos automatizados, comunicaciones de mando y control y otros comportamientos maliciosos persistentes. Este alto grado de realismo permite evaluar los modelos frente a patrones de ataque complejos y no triviales, más cercanos a situaciones reales de despliegue.

0.2.4.2 Estructura del dataset

IoT-23 se distribuye en forma de escenarios independientes, donde cada escenario corresponde a una captura concreta asociada a un tipo específico de malware o comportamiento malicioso. Estos escenarios se almacenan en directorios separados y presentan tamaños muy variables, desde unos pocos megabytes hasta decenas de gigabytes, lo que dificulta su uso directo en entornos experimentales con recursos limitados.

Con el fin de garantizar la viabilidad experimental y mantener la reproducibilidad de los experimentos, en este trabajo se ha seleccionado un subconjunto reducido de escenarios representativos. En concreto, se han utilizado los archivos `capture-1-1.labeled` y `capture-3-1.labeled`, que presentan un tamaño manejable y contienen tráfico IoT real con actividad maliciosa claramente identificable.

Los datos se proporcionan en forma de logs etiquetados generados por Zeek, con extensión `.labeled`. Cada registro representa un flujo de red e incluye características como direcciones IP, puertos, protocolo, duración de la conexión y métricas de tráfico, junto con información de etiquetado.

0.2.4.3 Distribución de ataques

La distribución de clases en los escenarios seleccionados de IoT-23 presenta un fuerte desbalanceo entre tráfico benigno y tráfico malicioso, así como una predominancia de determinados comportamientos de ataque característicos de dispositivos IoT comprometidos. En la Tabla 12 se muestra el número total de flujos correspondiente a cada clase y tipo de tráfico, considerando conjuntamente los archivos `capture-1-1.labeled` y `capture-3-1.labeled`.

Tabla 12. Distribución de clases y tipos de tráfico en el dataset IoT-23

Clase / Tipo de tráfico	Número de flujos
Malicious – PartOfAHorizontalPortScan	685 062
Benign	473 811
Malicious Attack	5 962
Malicious – C&C	16
Total	1 164 851

Como puede observarse, el tráfico malicioso asociado a escaneos horizontales de puertos domina el conjunto de datos, reflejando un patrón de ataque común en dispositivos IoT comprometidos. El tráfico benigno también constituye una parte significativa del dataset, mientras que otros tipos de ataques aparecen con menor frecuencia. Esta distribución permite evaluar la capacidad de los modelos para detectar patrones de ataque específicos en entornos IoT reales, donde la variabilidad y complejidad del tráfico pueden dificultar la detección efectiva.



Figura 11. Logo del Stratosphere Laboratory de la Czech Technical University, entidad responsable de la publicación del dataset IoT-23.

0.2.5 Relación entre los datasets

A continuación se muestra una tabla simplificada que resume la correspondencia principal entre las características de CIC-IDS-2017 y UNSW-NB15, facilitando la comparación y el diseño de modelos comunes.

Tabla 13. Correspondencia principal entre características de CIC-IDS-2017 y UNSW-NB15

Categoría	CIC-IDS-2017	UNSW-NB15
Duración flujo	Flow.Duration	dur
Intervalo paquetes origen	Fwd.IAT.Mean	sinpkt
Intervalo paquetes destino	Bwd.IAT.Mean	dinpkt
Paquetes origen	Total.Fwd.Packets	spkts
Paquetes destino	Total.Backward.Packets	dpkts
Bytes origen	Total.Length.of.Fwd.Packets	sbytes
Bytes destino	Total.Length.of.Bwd.Packets	dbytes
Tasa paquetes	Flow.Packets.s	rate
Tamaño medio paquetes origen	Fwd.Packet.Length.Mean	smean
Tamaño medio paquetes destino	Bwd.Packet.Length.Mean	dmean
SYN flag	SYN.Flag.Count	synack
ACK flag	ACK.Flag.Count	ackdat
Etiqueta	Label	label / attack_cat

Ahora relacionamos los datasets ToN_IoT y IoT-23, destacando los features comunes.

Tabla 14. Correspondencia principal entre características de ToN_IoT e IoT-23

Categoría	ToN_IoT	IoT-23	Descripción
IP origen	src_ip	id.orig_h	Dirección IP origen
Puerto origen	src_port	id.orig_p	Puerto origen
IP destino	dst_ip	id.resp_h	Dirección IP destino
Puerto destino	dst_port	id.resp_p	Puerto destino
Protocolo	proto	proto	Protocolo de transporte
Duración	duration	duration	Duración de la conexión
Bytes origen	src_bytes	orig_bytes	Bytes enviados por el origen
Bytes destino	dst_bytes	resp_bytes	Bytes enviados por el destino
Paquetes origen	src_pkts	orig_pkts	Paquetes enviados por el origen
Paquetes destino	dst_pkts	resp_pkts	Paquetes enviados por el destino
Estado	conn_state	conn_state	Estado de la conexión
Etiqueta	label	label	Tráfico benigno o malicioso
Tipo ataque	type	detailed-label	Tipo de ataque

0.2.6 Justificación del enfoque en la fase de inferencia

En el contexto de los Sistemas de Detección de Intrusiones (IDS) basados en inteligencia artificial, resulta fundamental distinguir entre la fase de entrenamiento del modelo y la fase de inferencia.

Aunque el entrenamiento puede implicar un coste computacional elevado, este proceso se realiza de manera puntual o con una frecuencia reducida. Por el contrario, la fase de inferencia se ejecuta de forma continua una vez que el modelo está desplegado en un entorno real, analizando de manera permanente el tráfico de red entrante.

El coste asociado a la inferencia es el que determina la viabilidad práctica del sistema. Un modelo con excelente rendimiento predictivo pero con alta latencia o consumo excesivo de CPU y memoria puede resultar inviable para su despliegue en tiempo real. Por este motivo, el presente trabajo centra su análisis experimental en el consumo computacional durante la fase de inferencia.

Desde la perspectiva de la Green AI, esta decisión está alineada con el objetivo de reducir el consumo energético en sistemas de uso continuo. Mientras que el entrenamiento representa un coste energético puntual, la inferencia constituye un coste acumulativo y sostenido en el tiempo. Por tanto, optimizar la eficiencia en esta fase tiene un impacto directo en la sostenibilidad del sistema IDS.

0.2.7 Optimización de hiperparámetros estructurales

Dado que el objetivo principal es analizar el equilibrio entre capacidad de detección y eficiencia computacional en inferencia, el ajuste de hiperparámetros se centra específicamente en aquellos que determinan la estructura del modelo. Los hiperparámetros estructurales influyen directamente en la complejidad computacional, el número de operaciones necesarias para generar una predicción y el tamaño final del modelo.

A diferencia de otros hiperparámetros relacionados con el proceso de entrenamiento (como la tasa de aprendizaje o el optimizador), los hiperparámetros estructurales determinan la arquitectura final del modelo y, por tanto, su coste en tiempo de ejecución una vez desplegado.

A continuación, se describen los hiperparámetros estructurales más relevantes para cada uno de los modelos considerados en este trabajo.

0.2.7.1 Random Forest

En el caso de Random Forest, el coste de inferencia depende principalmente de:

- **n_estimators**: número de árboles que componen el bosque. Un mayor número de árboles incrementa linealmente el número de recorridos necesarios para cada predicción.
- **max_depth**: profundidad máxima de cada árbol. Árboles más profundos implican más comparaciones por predicción.

0.2.7.2 XGBoost

En XGBoost, la inferencia consiste en recorrer secuencialmente todos los árboles entrenados y sumar sus contribuciones. Los hiperparámetros estructurales más relevantes son:

- **n_estimators**: número total de árboles del modelo.
- **max_depth**: profundidad máxima de cada árbol.

0.2.7.3 LightGBM

En LightGBM, el coste de inferencia también está influenciado por:

- **n_estimators**: número de árboles en el modelo.
- **max_depth**: profundidad máxima de cada árbol.
- **num_leaves**: número máximo de hojas por árbol, que afecta la complejidad del modelo y el número de comparaciones necesarias para cada predicción.

0.2.7.4 CatBoost

En CatBoost, los hiperparámetros estructurales que afectan el coste de inferencia son:

- **iterations**: número de árboles a entrenar. Un mayor número de iteraciones incrementa linealmente el coste computacional en inferencia, ya que cada predicción requiere recorrer todos los árboles.
- **depth**: profundidad máxima de cada árbol. Árboles más profundos implican más comparaciones y operaciones por predicción.

0.2.7.5 Support Vector Machine (SVM)

Para SVM, el coste de inferencia depende principalmente del tipo de kernel utilizado. En este trabajo se prioriza el uso de:

- **kernel lineal**: permite realizar predicciones mediante un producto escalar, lo que reduce significativamente el coste computacional.
- **C**: parámetro de regularización que puede influir en la complejidad del modelo, aunque su impacto en la inferencia es menor que el tipo de kernel.

0.2.7.6 Multilayer Perceptron (MLP)

En redes MLP, el coste de inferencia está directamente relacionado con el número total de pesos del modelo. Los hiperparámetros estructurales clave son:

- **hidden_layer_sizes**: número de capas ocultas y número de neuronas por capa.

0.2.7.7 Long Short-Term Memory (LSTM)

En las redes LSTM, la complejidad de inferencia depende de:

- **lstm_units**: número de unidades LSTM en la capa recurrente.
- **time_steps**: número de pasos temporales procesados.
- **num_layers**: número de capas en la red.

0.2.7.8 Convolutional Neural Networks (CNN 1D)

En las CNN 1D utilizadas en este trabajo, los hiperparámetros estructurales más relevantes son:

- **n_filters**: número de filtros en las capas convolucionales.
- **kernel_size**: tamaño del kernel.
- **dense_units**: número de unidades en la capa densa.
- **time_steps**: número de pasos temporales procesados.

0.3 Optuna

Optuna es un framework de optimización de hiperparámetros de código abierto que permite a los desarrolladores y científicos de datos encontrar automáticamente los mejores hiperparámetros para sus modelos de aprendizaje automático. Optuna utiliza técnicas de optimización bayesiana para explorar el espacio de hiperparámetros de manera eficiente, lo que puede mejorar significativamente el rendimiento de los modelos.

A diferencia de Grid Search, que realiza una búsqueda exhaustiva y costosa, o Random Search, que no aprende de iteraciones pasadas, Optuna utiliza un enfoque bayesiano basado en el Tree-structured Parzen Estimator (TPE).

El algoritmo TPE construye un modelo probabilístico de la función objetivo. En lugar de probar combinaciones al azar, predice qué regiones del espacio de búsqueda tienen más probabilidades de mejorar los resultados actuales.

Otra de las ventajas de este framework es que nos permite establecer una optimización multiobjetivo, lo que es especialmente útil cuando se desea optimizar varias métricas al mismo tiempo, como la precisión y eficiencia del modelo.

Optuna identifica un conjunto de soluciones que se encuentran en la frontera de Pareto, lo que significa que no hay otra solución que sea mejor en todas las métricas. Esto permite a los usuarios elegir entre

diferentes compromisos según sus necesidades específicas, por lo que no presentamos una única solución.



Figura 12. Visualización de la optimización con Optuna

0.4 Resultados

0.4.1 Entorno de pruebas

Antes de analizar los resultados obtenidos, es importante describir el entorno de pruebas utilizado para la evaluación de los modelos. Todos los experimentos se han llevado a cabo en un entorno controlado con las siguientes características:

Componente	Especificación Técnica
Sistema Operativo	Debian GNU/Linux 12 (bookworm)
Procesador (CPU)	2x Intel® Xeon® Gold 5418Y (Arquitectura Dual-Socket)
Núcleos de CPU	48 físicos / 96 lógicos (Hilos)
Memoria RAM (Sistema)	512 GB (503 GiB disponibles)
Aceleradores Gráficos (GPU)	4x NVIDIA L40S
Memoria VRAM (Total)	192 GB (48 GB dedicados por cada GPU)
Plataforma de Cómputo	CUDA Version 13.0 (Driver 580.95.05)

Tabla 15. Especificaciones del entorno experimental

0.4.2 Modelos entrenados con UNSW-NB15

0.4.2.1 Random Forest

Para el modelo de Random Forest, utilizando Optuna, se han obtenido los siguientes resultados como frontera de Pareto:

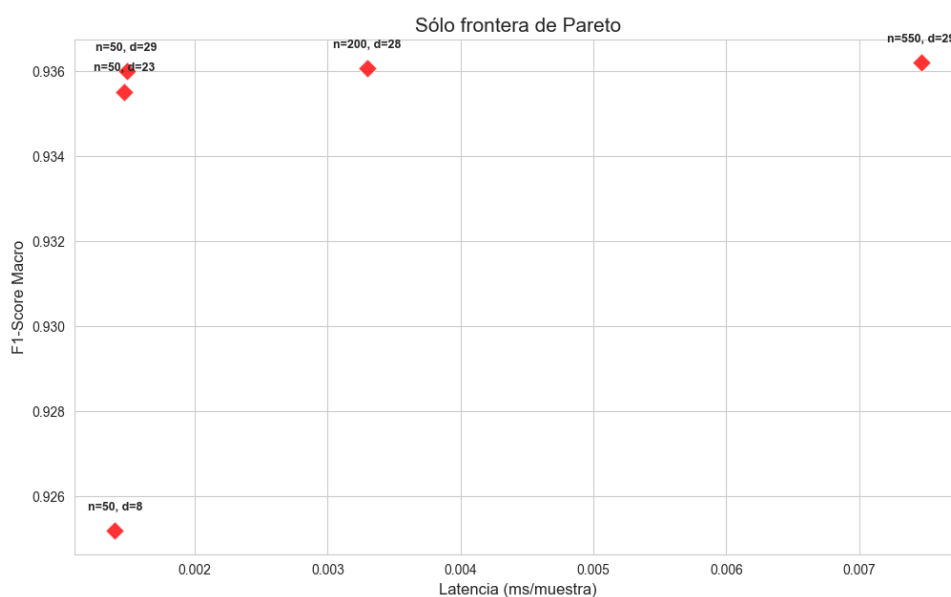


Figura 13. Frontera de Pareto - Random Forest

Hemos visto cómo reaccionan en test los modelos con los siguientes hiperparámetros elegidos:

1. (50, 23)
2. (100, 29)
3. (200, 28)
4. (550, 29)

Los resultados son los siguientes:

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	23	0.885392	0.888622	0.001516
Candidato 2	50	29	0.873085	0.877411	0.001486
Candidato 3	200	28	0.876276	0.880338	0.003443
Candidato 4	550	29	0.87463	0.878881	0.007078

Tabla 16. Comparativa final de modelos Random Forest en el set de test

El **Candidato 1** no solo es el más preciso, sino también uno de los más rápidos, con una latencia de 0,0015 ms por muestra. Con esto podemos ver que modelos más sencillos de árboles ofrecen hasta mayor rendimiento que modelos más complejos, lo que es un resultado muy interesante para el desarrollo de un IDS eficiente y sostenible.

0.4.2.2 XGBoost

En XGBoost primero optamos por un entrenamiento con CPU. Sin embargo, debido a la elevada latencia obtenida en la frontera de Pareto, se decidió realizar un nuevo entrenamiento utilizando GPU, lo que permitió reducir significativamente los tiempos de inferencia. A continuación se muestran los resultados obtenidos:

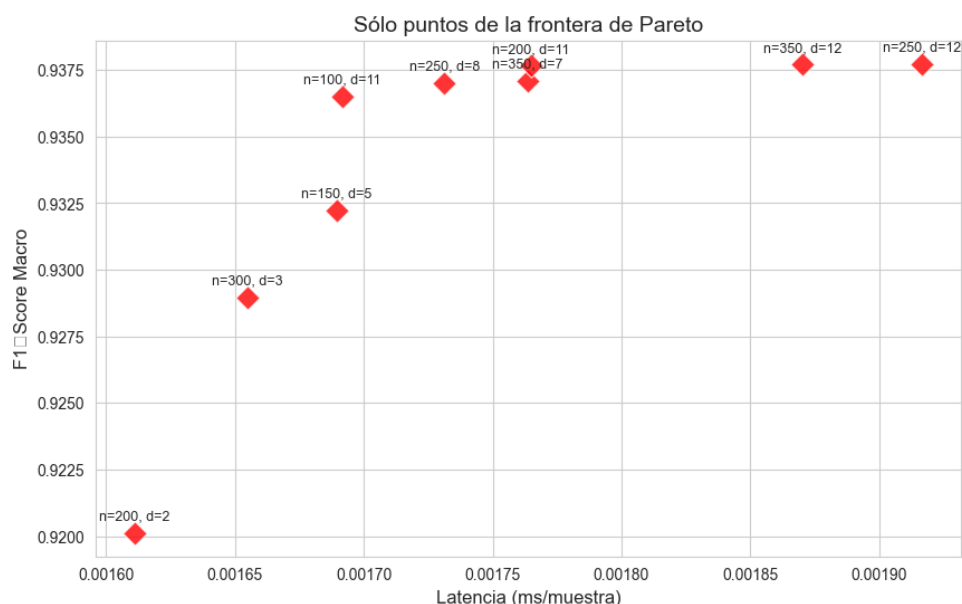


Figura 14. Frontera de Pareto - XGBoost

Hemos seleccionado las 3 combinaciones de la frontera de Pareto para testarlas:

1. (350, 12)
2. (200, 11)
3. (350, 7)
4. (250, 8)
5. (100, 11)

Los resultados son los siguientes:

Un hallazgo fundamental de este experimento es la diferencia de topología requerida entre algoritmos. Mientras que Random Forest necesitaba árboles extremadamente profundos (profundidades de 23 a 29) para capturar las anomalías, XGBoost alcanza su máximo rendimiento con profundidades mucho menores ($d=11$ o $d=12$). Esto demuestra teóricamente la eficiencia del Gradient Boosting: al construir árboles de forma secuencial (donde cada árbol corrige los errores del anterior), el modelo no necesita que cada árbol individual sea excesivamente complejo y profundo. Nos quedamos con el **Candidato 2**, que ofrece un excelente equilibrio entre calidad y coste.

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	350	12	0.858299	0.864378	0.000501
Candidato 2	200	11	0.857905	0.864111	0.000418
Candidato 3	350	7	0.856623	0.863152	0.000561
Candidato 4	250	8	0.856923	0.863322	0.000543
Candidato 5	100	11	0.857487	0.86388	0.000443

Tabla 17. Comparativa final de modelos XGBoost en el set de test

0.4.2.3 LightGBM

LightGBM ha sido entrenado con GPU, obteniendo la siguiente frontera de Pareto:

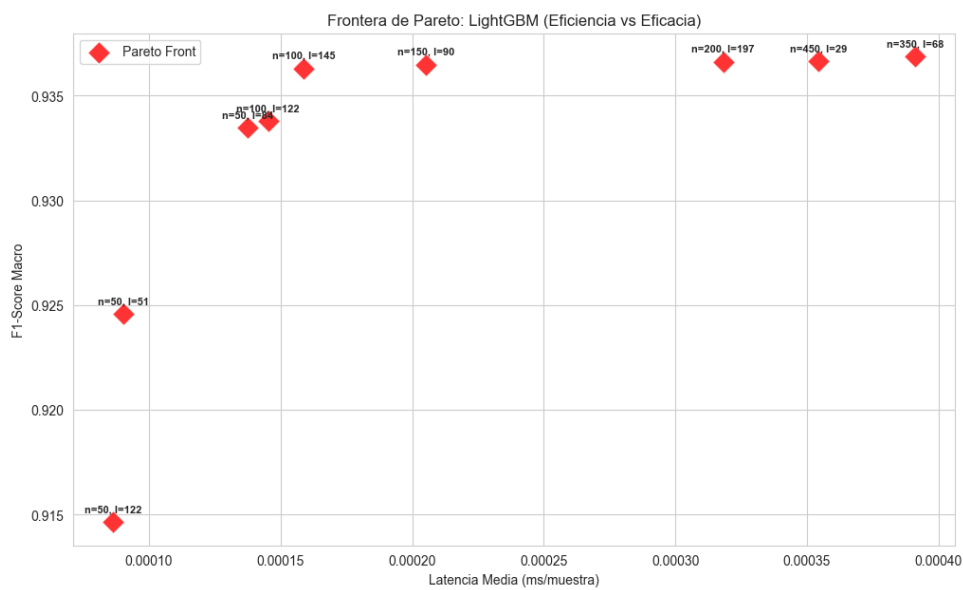


Figura 15. Frontera de Pareto - LightGBM

Entre los diferentes puntos de la frontera de Pareto, se han seleccionado los siguientes para su evaluación en el set de test:

1. (350, 68, 9)
2. (450, 29, 12)
3. (150, 90, 12)
4. (100, 145, 12)
5. (100, 122, 7)

Los resultados obtenidos en el set de test son los siguientes:

Perfil	n_estimators	num_leaves	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	350	68	9	0.861467	0.867354	0.000371
Candidato 2	450	29	12	0.860196	0.86631	0.00036
Candidato 3	150	90	12	0.860457	0.866552	0.0002
Candidato 4	100	145	12	0.859564	0.865775	0.000159
Candidato 5	100	122	7	0.852491	0.859654	0.000137

Tabla 18. Comparativa final de modelos LightGBM en el set de test

Si tuviéramos que quedarnos con un único modelo, nos quedamos con el **Candidato 4**. Al permitir 145 hojas, el algoritmo puede modelar reglas muy complejas del tráfico de red en cada iteración, lo que le permite necesitar solo 100 árboles para aprender casi lo mismo que el Candidato 1 aprende en 350. Esto se traduce en un F1-Score del 85.95% (reteniendo el 99.7% de la máxima eficacia posible) pero con una latencia de 0.000159 ms.

0.4.2.4 CatBoost

CatBoost ha sido entrenado utilizando GPU, obteniendo la siguiente frontera de Pareto:

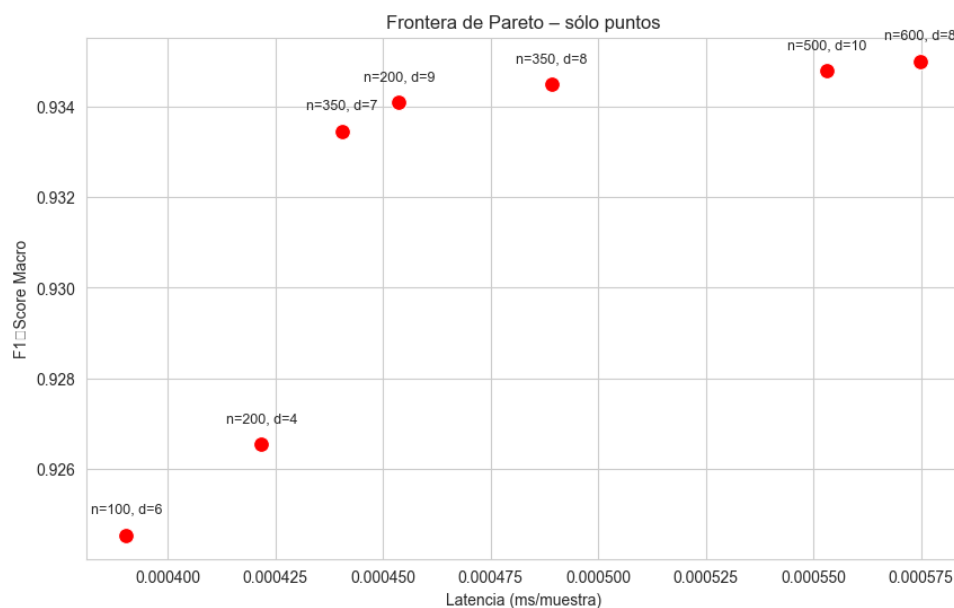


Figura 16. Frontera de Pareto - CatBoost

Entre los diferentes puntos de la frontera de Pareto, se han seleccionado los siguientes para su evaluación en el set de test:

1. (200, 4)
2. (350, 7)
3. (200, 9)

4. (350, 8)

5. (500, 10)

Los resultados obtenidos en el set de test son los siguientes:

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	200	4	0.828623	0.838957	0.000298
Candidato 2	350	7	0.854301	0.861135	0.00044
Candidato 3	200	9	0.85276	0.859787	0.000477
Candidato 4	350	8	0.855648	0.862277	0.000489
Candidato 5	500	10	0.85702	0.863419	0.000375

Tabla 19. Comparativa final de modelos CatBoost en el set de test

El dato más revelador de la tabla es la latencia del **Candidato 5** ($n=500$, $d=10$). A pesar de ser el modelo más masivo y profundo de los evaluados, presenta una latencia de 0.000375 ms, siendo más rápido que modelos teóricamente más sencillos como el Candidato 4 ($n=350$, $d=8$, con 0.000489 ms). Esto se explica por el uso de oblivious trees (árboles simétricos ajenos) en la arquitectura interna de CatBoost. En inferencia, evaluar estos árboles se reduce a simples operaciones de bits a nivel de procesador, lo que significa que evaluar 500 árboles en CatBoost puede ser más rápido que evaluar 350 si la estructura encaja mejor en la memoria caché de la CPU.

0.4.2.5 Support Vector Machine (SVM)

La gráfica de la frontera de Pareto obtenida para SVM:

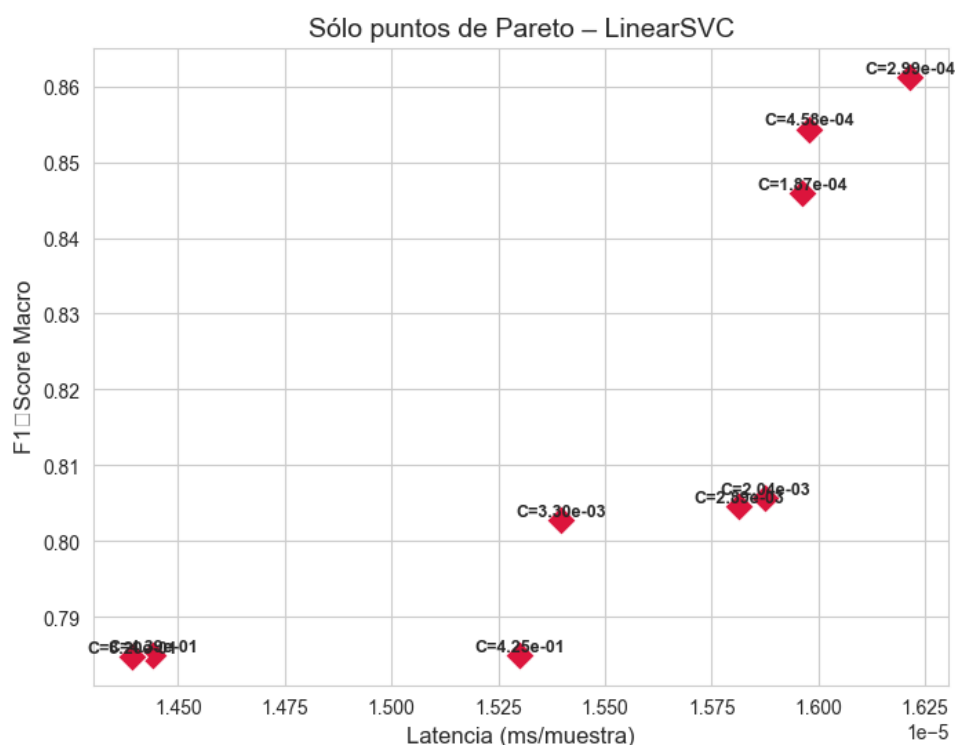


Figura 17. Frontera de Pareto – SVM

Los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto son los siguientes:

Perfil	C	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	0.000299	0.708654	0.735461	0.000081
Candidato 2	0.000458	0.692147	0.722987	0.000093
Candidato 3	0.000187	0.709219	0.736105	0.000018
Candidato 4	0.002037	0.649204	0.692003	0.000019
Candidato 5	0.002889	0.648003	0.691104	0.000018

Tabla 20. Comparativa final de modelos SVM en el set de test

A diferencia de los modelos basados en árboles (RF, XGBoost, LightGBM) que superaban holgadamente el 85% de F1-Score, LinearSVC se estanca en un máximo de 0.7092 (Candidato 3). Esta drástica caída de rendimiento demuestra empíricamente que las intrusiones de red no son linealmente separables. Un simple hiperplano multidimensional es insuficiente para trazar la frontera de decisión entre el tráfico normal y los ataques, validando la necesidad de utilizar modelos no lineales más complejos para este dominio.

Dentro de las limitaciones del enfoque lineal, el **Candidato 3** ($C=0.000187$) es el ganador absoluto e indiscutible.

0.4.2.6 Multilayer Perceptron (MLP)

La gráfica de la frontera de Pareto obtenida para MLP es la siguiente:

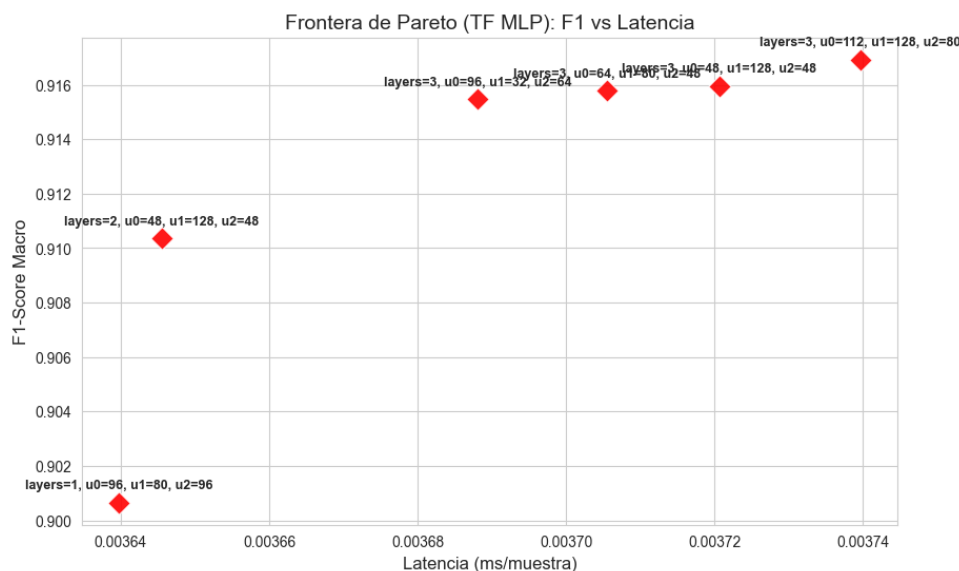


Figura 18. Frontera de Pareto - MLP

Ahora mostramos los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto:

Perfil	n_layers	n_units_l0	n_units_l1	n_units_l2	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	3	96	80	96	0.777448	0.797017	0.00159
Candidato 2	3	48	128	48	0.758004	0.781397	0.001505
Candidato 3	3	64	80	48	0.782289	0.800515	0.001505
Candidato 4	3	96	32	64	0.786982	0.804147	0.001421

Tabla 21. Comparativa final de modelos MLP en el set de test

Nos quedamos con el **Candidato 4**, que ofrece el mejor rendimiento (F1-Score de 0.7869) y una latencia de inferencia de 0.001421 ms, lo que lo posiciona como un modelo competitivo dentro de los modelos no lineales, aunque aún por debajo de los basados en árboles. Además hemos visto que menos neuronas en las capas tienen menor tiempo de inferencia, lo que es un resultado lógico, ya que a menor número de unidades, menor cantidad de operaciones matemáticas se deben realizar para obtener la predicción, lo que se traduce en una reducción de la latencia.

0.4.2.7 LSTM

Con LSTM, la gráfica de la frontera de Pareto obtenida es la siguiente:

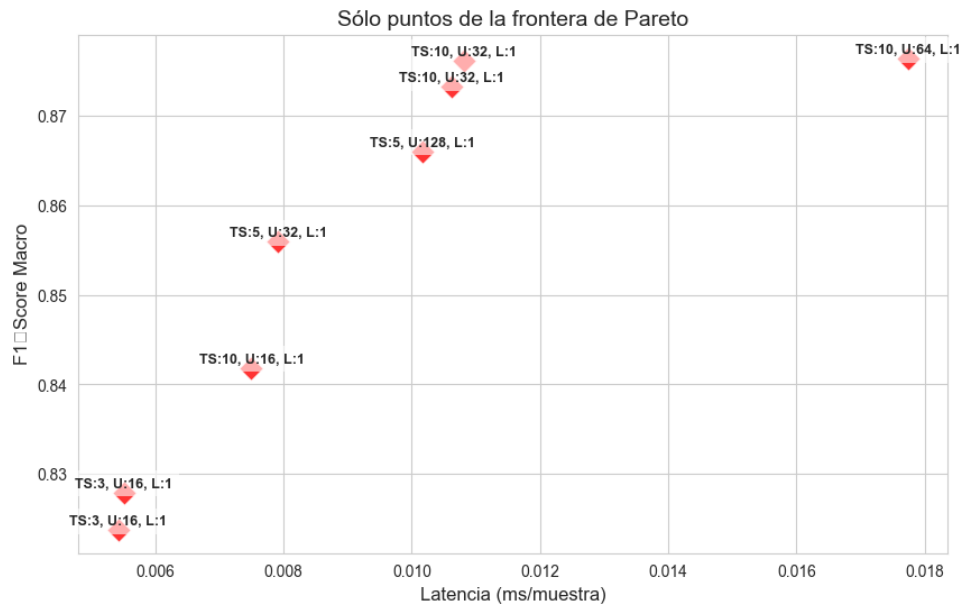


Figura 19. Frontera de Pareto - LSTM

Los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto son los siguientes:

Perfil	TS	Units	Layers	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	10	32	1	0.918214	0.920569	0.008549
Candidato 2	5	128	1	0.893188	0.897131	0.007129
Candidato 3	5	32	1	0.886394	0.890766	0.004702
Candidato 4	10	64	1	0.913959	0.916597	0.014846

Tabla 22. Comparativa final de modelos LSTM en el set de test

Nos quedamos con el **Candidato 1**, que ofrece un excelente rendimiento (F1-Score de 0.9182) pero con una latencia de inferencia de 0.008549 ms, lo que lo posiciona como un modelo muy preciso pero con un coste computacional elevado si lo comparamos con los modelos de árboles. Los modelos con TS: 10 (Candidatos 1 y 4) superan holgadamente la barrera del 0.91 de F1. Los modelos con TS: 5 (Candidatos 2 y 3) se quedan en el rango de 0.88 - 0.89. Incluso si le das 128 unidades a un modelo con TS:5 (Candidato 2), no logra alcanzar el rendimiento de un modelo con solo 32 unidades pero con TS:10. Conclusión: El modelo necesita "ver" más atrás en el tiempo (TS:10) para entender correctamente el patrón, y añadir más "neuronas" (unidades) no compensa la falta de contexto temporal.

0.4.2.8 CNN

Aunque CNN se suele asociar principalmente a datos con estructura espacial (como imágenes), también se ha explorado su aplicación en la detección de intrusiones. Esta es la gráfica de la frontera de Pareto obtenida para CNN:

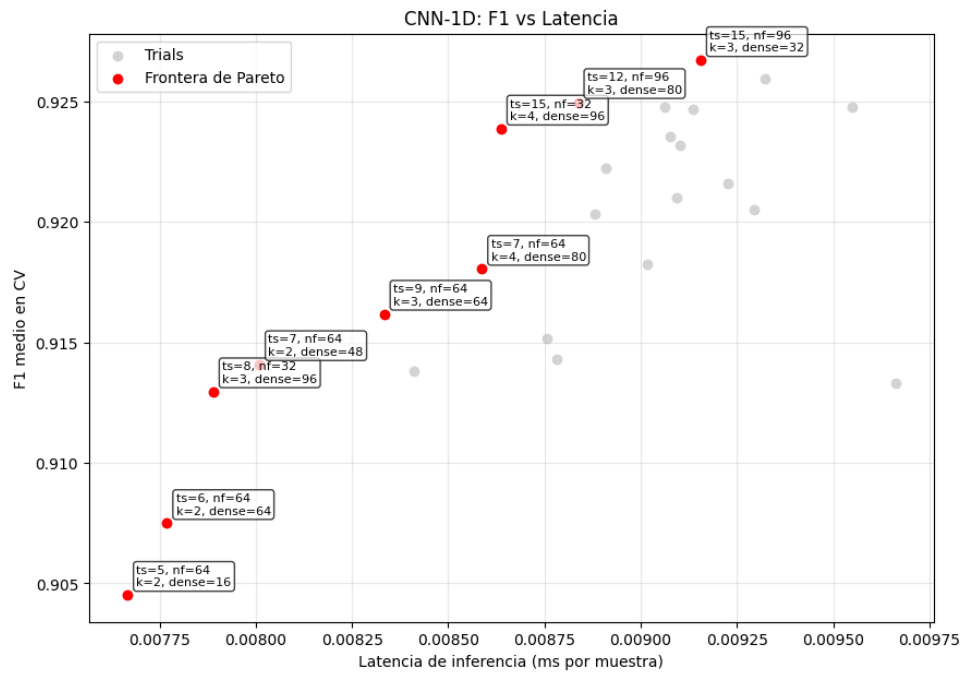


Figura 20. Frontera de Pareto - CNN

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

Perfil	time_steps	n_filters	kernel_size	dense_units	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	15	96	3	32	0.919338	0.932591	0.009589
Candidato 2	15	32	4	96	0.907723	0.923553	0.009057
Candidato 3	12	96	3	80	0.905609	0.921806	0.009613
Candidato 4	7	64	4	80	0.897475	0.915069	0.008658
Candidato 5	9	64	3	64	0.892452	0.912091	0.008549

Tabla 23. Comparativa final de modelos CNN en el set de test

El **Candidato 1** es el modelo ganador, con un F1-Score de 0.9193 y una latencia de inferencia de 0.009589 ms. Aunque su rendimiento es excelente, su latencia es significativamente mayor que la de los modelos basados en árboles, lo que podría limitar su aplicabilidad en entornos de producción donde la velocidad de detección es crítica.

0.4.2.9 Comparativa Modelos con Dataset UNSW-NB15

No solo hemos comparado los modelos a través de su F1-Score y latencia, sino que también hemos realizado una evaluación global de su rendimiento en términos de latencia, throughput, uso de CPU y RAM. Para calcular las métricas de rendimiento, hemos usado los siguientes conceptos:

Métrica	Descripción	Metodología
Tiempo de Muro	Tiempo físico y real que transcurre desde el inicio hasta la finalización. Métrica definitiva de la latencia percibida por el usuario o la red. Independientemente de cuántos núcleos trabaje el servidor, es el tiempo exacto que un paquete debe esperar para ser clasificado.	Obtenido utilizando <code>time.perf_counter()</code> , que proporciona el reloj de mayor resolución disponible en el hardware, inmune a actualizaciones automáticas de la hora del sistema.
Tiempo de CPU	Suma total del tiempo que la Unidad Central de Procesamiento ejecuta activamente instrucciones. Acumula el esfuerzo de todos los hilos simultáneamente . Si el Tiempo de Muro es 1 segundo pero Tiempo de CPU es 95 segundos, indica que 95 hilos lógicos trabajaron en paralelo a máxima capacidad.	Obtenido mediante <code>psutil.Process().cpu_times()</code> , sumando tiempo de usuario y llamadas al sistema. Se calcula la diferencia (delta) entre inicio y fin del lote.
Pico de RAM	Porción de memoria física (RAM) asignada de manera exclusiva al proceso y residente activamente en memoria. Algoritmos como CNN generan tensores intermedios masivos que requieren picos muy elevados, pudiendo inviabilizar despliegue en hardware de frontera con memoria limitada.	Sondeo iterativo mediante <code>psutil.Process().memory_info().rss</code> . Se captura la memoria base antes de comenzar y se resta al valor máximo registrado durante procesamiento de lotes.

Tabla 24. Definición técnica de métricas de monitoreo de recursos

Métrica	Significado Técnico	Metodología de Código
F1-Score	Métrica de eficacia predictiva. Es la media armónica entre Precisión y Sensibilidad (Recall). Representa la capacidad del modelo para detectar ataques reales minimizando las falsas alarmas, siendo ideal para tráfico desbalanceado.	Calculada mediante <code>scikit-learn (f1_score)</code> , cruzando las etiquetas reales del dataset de test frente a las predicciones generadas por el modelo.
Latencia (ms)	Tiempo de respuesta unitario. Representa los milisegundos que tarda el sistema en procesar y clasificar un único paquete desde entrada hasta veredicto.	Medida mediante <code>time.perf_counter()</code> al procesar el set de test por lotes, dividido entre el número total de paquetes procesados.
Throughput (paq/s)	Caudal de procesamiento. Representa el volumen de tráfico (paquetes por segundo) que el modelo puede soportar sin cuellos de botella.	Dividiendo el número total de paquetes del conjunto de test entre el tiempo físico total (Wall Time en segundos).
Núcleos CPU	Nivel de concurrencia y paralelismo. Representa la cantidad de hilos simultáneos que el modelo ha instanciado a nivel del sistema operativo.	Dividiendo el Tiempo Total de CPU (obtenido vía <code>psutil</code>) entre el tiempo físico real transcurrido (Wall Time).
Pico RAM (MB)	Huella máxima de memoria volátil. Representa la cantidad de memoria física extra que el sistema operativo cedió al proceso durante la predicción.	Monitorizando el proceso con <code>psutil.memory_info().rss</code> , restando la memoria base al pico máximo alcanzado.
Porcentaje CPU (%)	Carga computacional global. Representa qué porcentaje de la capacidad total del hardware está siendo utilizado por el IDS.	Dividiendo los Núcleos CPU utilizados entre el número total de hilos lógicos disponibles en el servidor.

Tabla 25. Definición de métricas de evaluación de rendimiento

Tras hacer una fase de benchmarking con los mejores candidatos de los diferentes modelos, hemos obtenido los siguientes resultados:

Tabla 26. Tabla global de rendimiento de hardware

Modelo	Latencia (ms)	Throughput (paq/s)	Núcleos CPU	Pico RAM (MB)	Porcentaje CPU (%)
Linear SVM	0.00014	7.18206×10^6	1.2	0.00	1.2
LightGBM	0.00064	1.574704×10^6	95.3	0.02	99.3
XGBoost	0.00066	1.522112×10^6	96.8	0.00	100.8
CatBoost	0.00150	664638	48.3	0.20	50.3
Random Forest	0.00893	112034	1.5	7.18	1.6
TensorFlow MLP	0.02541	39352	1.4	6.08	1.5
CNN-1D	0.05279	18943	2.0	189.15	2.1
LSTM	0.05521	18113	1.5	7.91	1.6

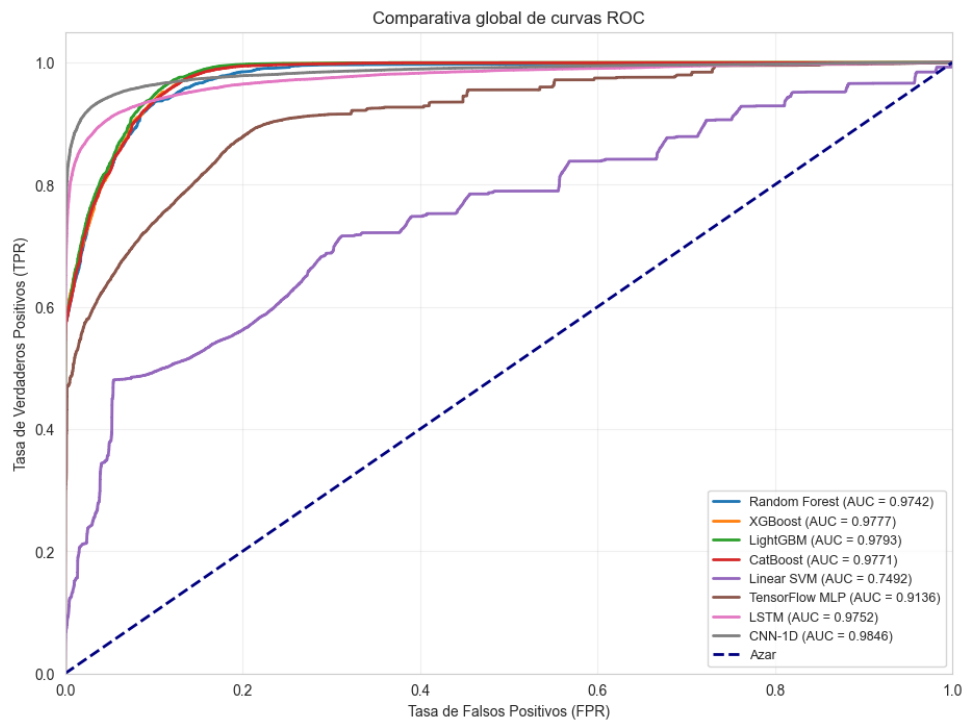


Figura 21. Curva ROC - Modelos

Las matrices de confusión de cada modelo se muestran a continuación:

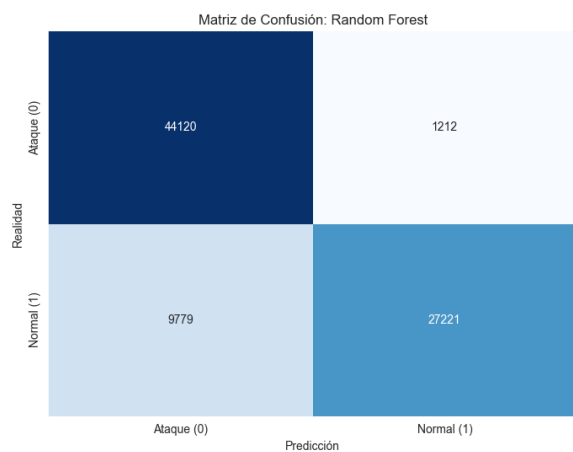


Figura 22. MC - Random Forest

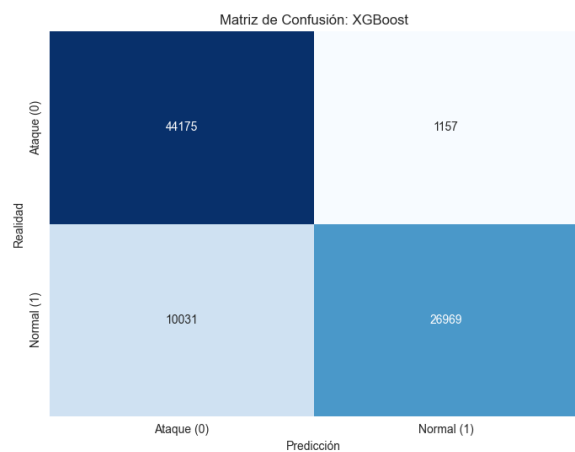


Figura 23. MC - XGBoost

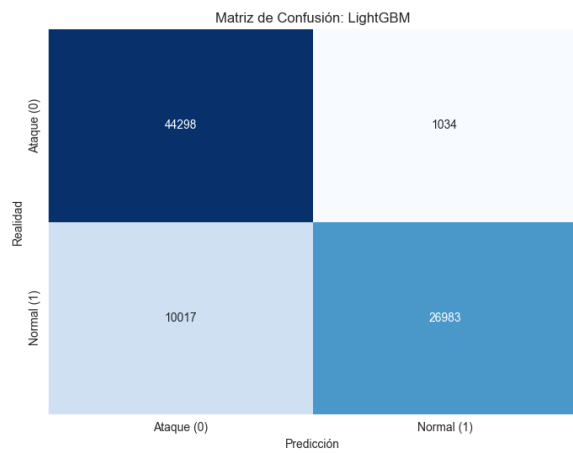


Figura 24. MC - LightGBM

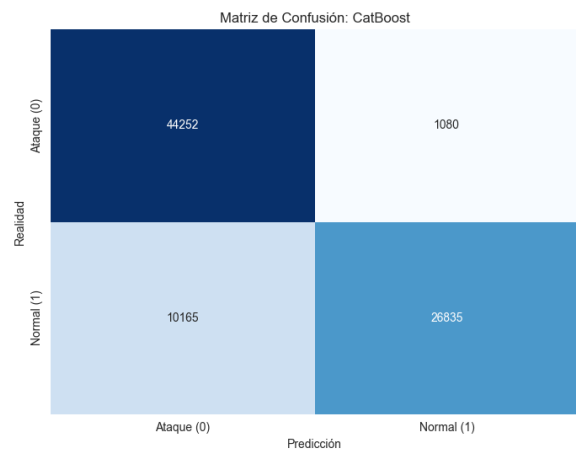


Figura 25. MC - CatBoost

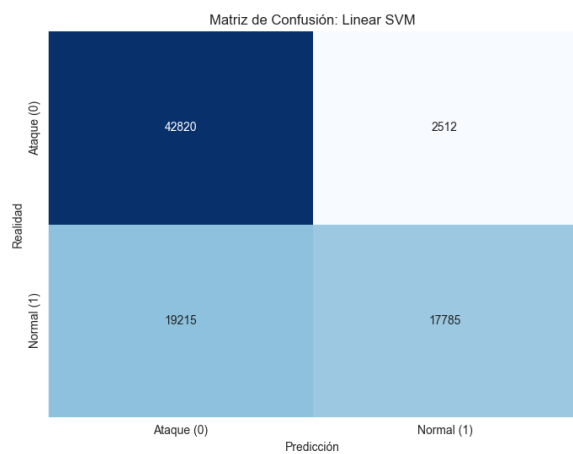


Figura 26. MC - SVM

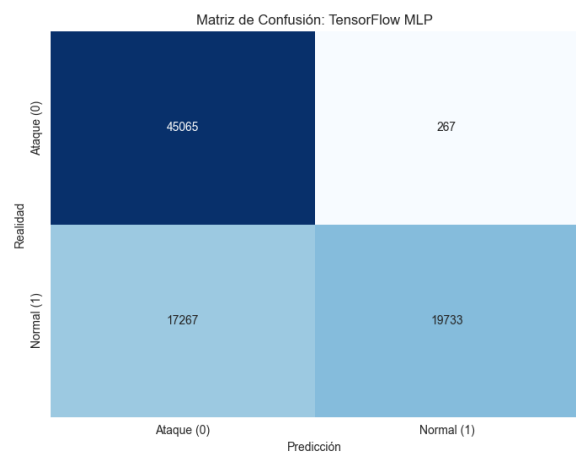


Figura 27. MC - MLP

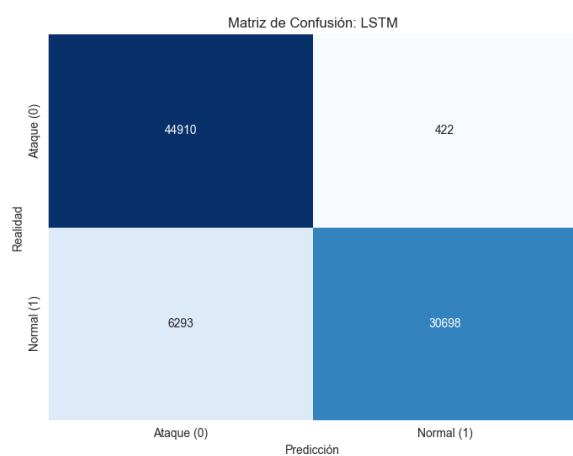


Figura 28. MC - LSTM

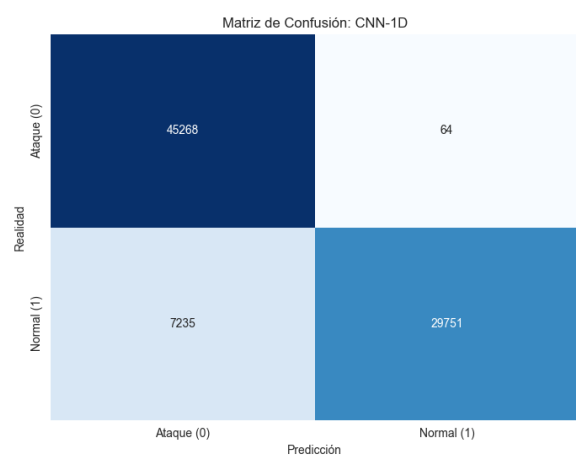


Figura 29. MC - CNN

0.4.3 Modelos entrenados con IOT-23

Para completar el análisis y las comparaciones, es interesante ver cómo se comportan los modelos con un dataset adaptado a un entorno de Internet de las Cosas (IoT), que suele presentar características muy diferentes a los datasets tradicionales de tráfico de red. En este caso, se han entrenado los mismos modelos que el apartado anterior, pero usando este dataset.

0.4.3.1 Random Forest

Para el modelo de Random Forest, utilizando Optuna, se han obtenido los siguientes resultados como frontera de Pareto:

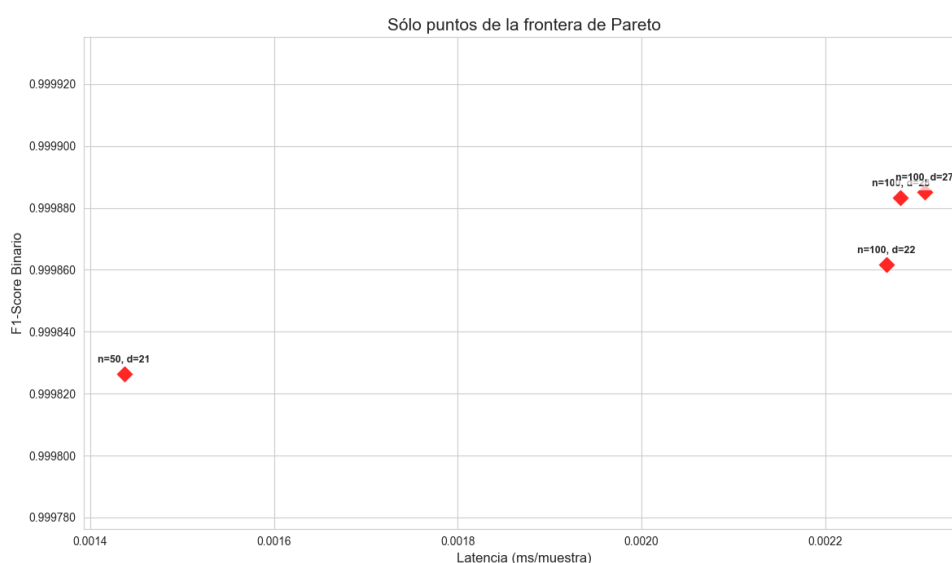


Figura 30. Frontera de Pareto – Random Forest (IOT-23)

Hemos visto cómo reaccionan en test los modelos con los siguientes hiperparámetros elegidos:

1. (50, 21)
2. (100, 22)
3. (100, 28)
4. (100, 27)

Los resultados son los siguientes:

En este caso, el **Candidato 1** es el modelo más eficiente, con un F1-Score de 0.999675 y una latencia de inferencia de 0.001496 ms, lo que lo posiciona como un modelo extremadamente preciso y rápido para este dataset específico de IoT.

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	21	0.999675	0.999614	0.001496
Candidato 2	100	22	0.999855	0.999828	0.002252
Candidato 3	100	28	0.999906	0.999888	0.002363
Candidato 4	100	27	0.999913	0.999897	0.00223

Tabla 27. Comparativa final de modelos Random Forest en el set de test (IOT-23)

0.4.3.2 XGBoost

Para XGBoost, la frontera de Pareto obtenida es la siguiente:

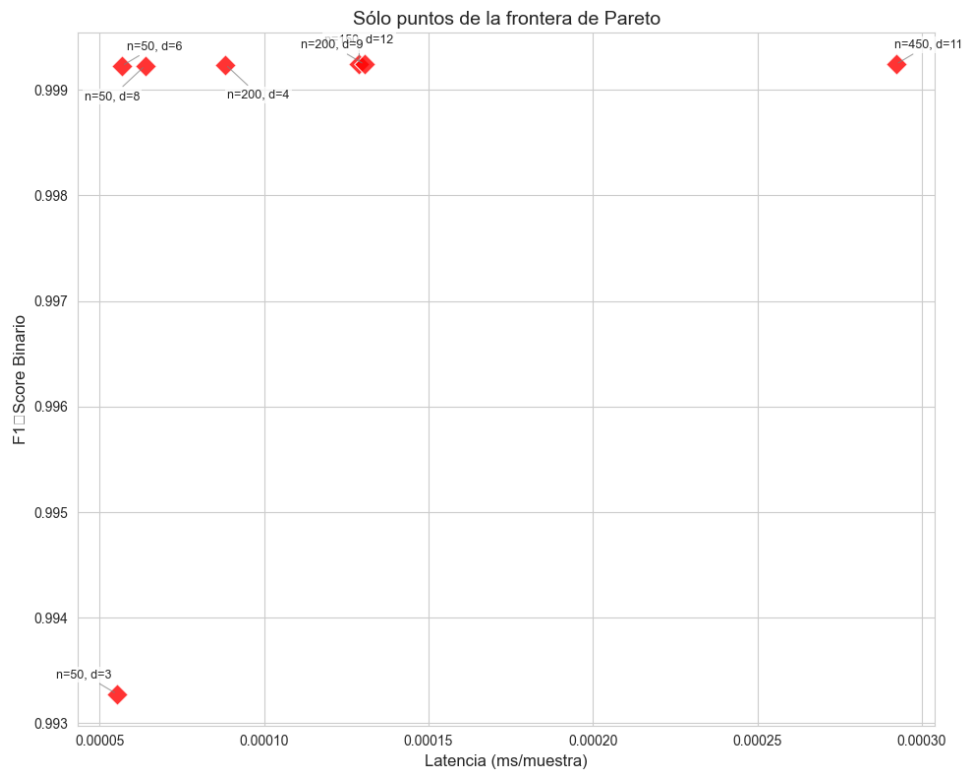


Figura 31. Frontera de Pareto - XGBoost (IOT-23)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (50, 8)
2. (50, 6)
3. (200, 4)
4. (200, 9)
5. (150, 12)

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	8	0.999114	0.998948	0.000028
Candidato 2	50	6	0.999122	0.998957	0.000027
Candidato 3	200	4	0.999122	0.998957	0.000047
Candidato 4	200	9	0.999132	0.998970	0.000080
Candidato 5	150	12	0.999132	0.998970	0.000080

Tabla 28. Comparativa final de modelos XGBoost en el set de test (IOT-23)

En este caso, el **Candidato 2** es el modelo más eficiente, con un F1-Score de 0.999122 y una latencia de inferencia de 0.000027 ms, lo que lo posiciona como un modelo extremadamente preciso y rápido en este dataset de IoT, superando incluso a los modelos de Random Forest en términos de latencia.

0.4.3.3 LightGBM

La frontera de Pareto obtenida para LightGBM es la siguiente:

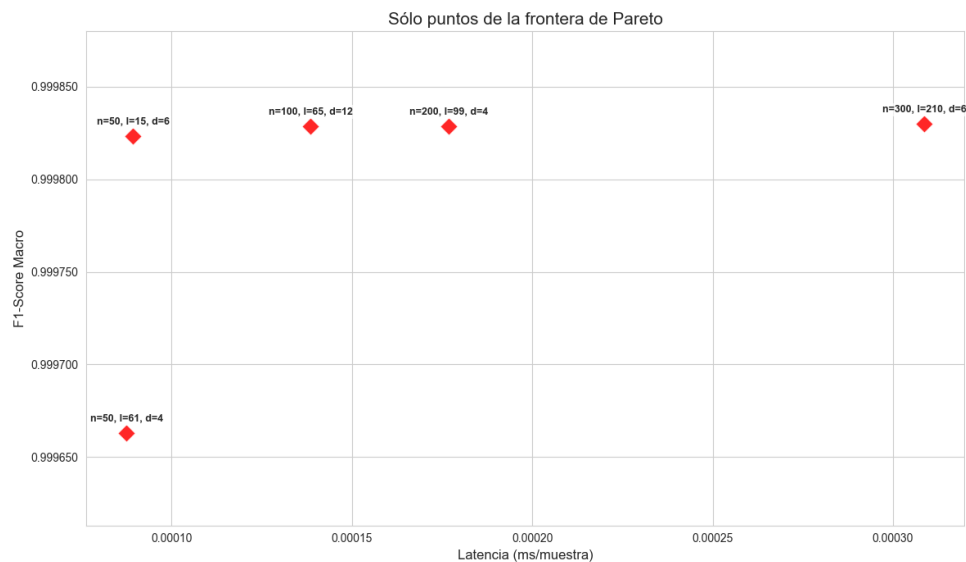


Figura 32. Frontera de Pareto - LightGBM (IOT-23)

Para este modelo, se han seleccionado los siguientes puntos de la frontera de Pareto para su evaluación en el set de test:

1. (50, 15, 6)
2. (100, 65, 12)
3. (200, 99, 4)
4. (50, 61, 4)
5. (300, 210, 6)

Los resultados obtenidos en el set de test son los siguientes:

Perfil	n_estimators	num_leaves	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	15	6	0.999813	0.99982	0.000057
Candidato 2	100	65	12	0.999831	0.999837	0.000099
Candidato 3	200	99	4	0.999795	0.999803	0.000170
Candidato 4	50	61	4	0.999631	0.999644	0.000058
Candidato 5	300	210	6	0.881924	0.882891	0.000261

Tabla 29. Comparativa final de modelos LightGBM en el set de test (IOT-23)

En este caso, el **Candidato 1** es el mejor modelo, con un F1-Score de 0.999813 y una latencia de inferencia de 0.000057 ms, lo que lo posiciona como un modelo bastante bueno.

0.4.3.4 CatBoost

Este es el gráfico de la frontera de Pareto obtenida para CatBoost:

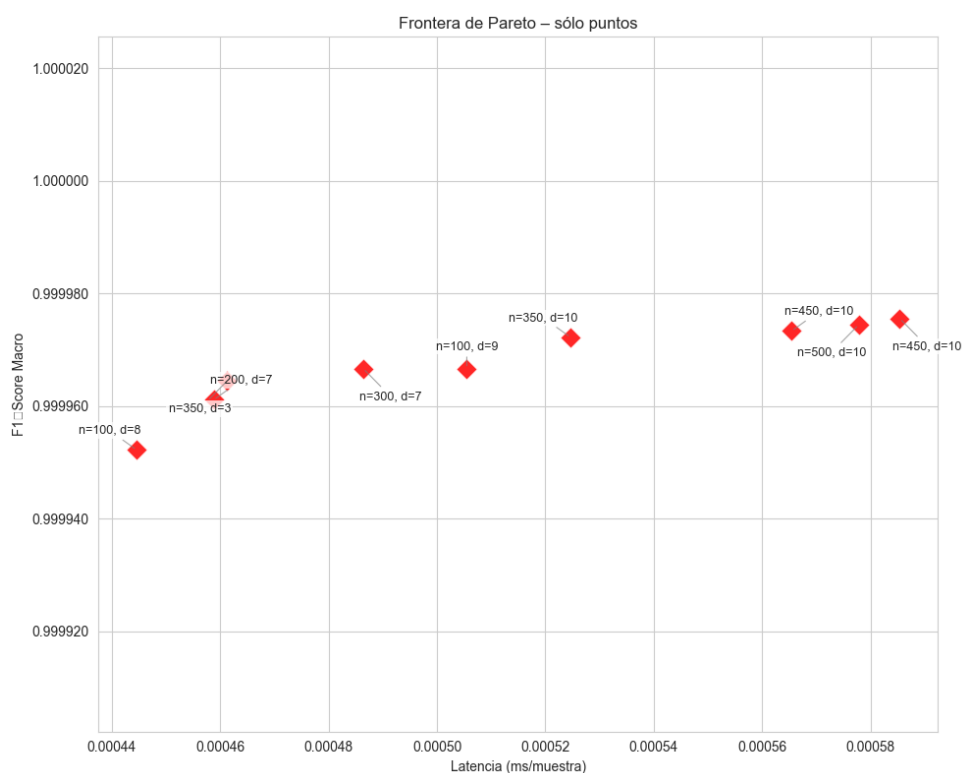


Figura 33. Frontera de Pareto - CatBoost (IOT-23)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (100, 8)
2. (350, 3)

3. (200, 7)
4. (300, 7)
5. (100, 9)
6. (350, 10)

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	100	8	0.999947	0.999948	0.000242
Candidato 2	350	3	0.999960	0.999961	0.000256
Candidato 3	200	7	0.999951	0.999953	0.000263
Candidato 4	300	7	0.999951	0.999953	0.000250
Candidato 5	100	9	0.999951	0.999953	0.000259
Candidato 6	350	10	0.999951	0.999953	0.000269

Tabla 30. Comparativa final de modelos CatBoost en el set de test (IOT-23)

En este caso, el **Candidato 2** es el modelo más eficiente, con un F1-Score de 0.999960 y una latencia de 0.000256 ms, lo que lo posiciona como un modelo extremadamente bueno.

0.4.3.5 SVM

La gráfica de la frontera de Pareto obtenida para SVM es la siguiente:

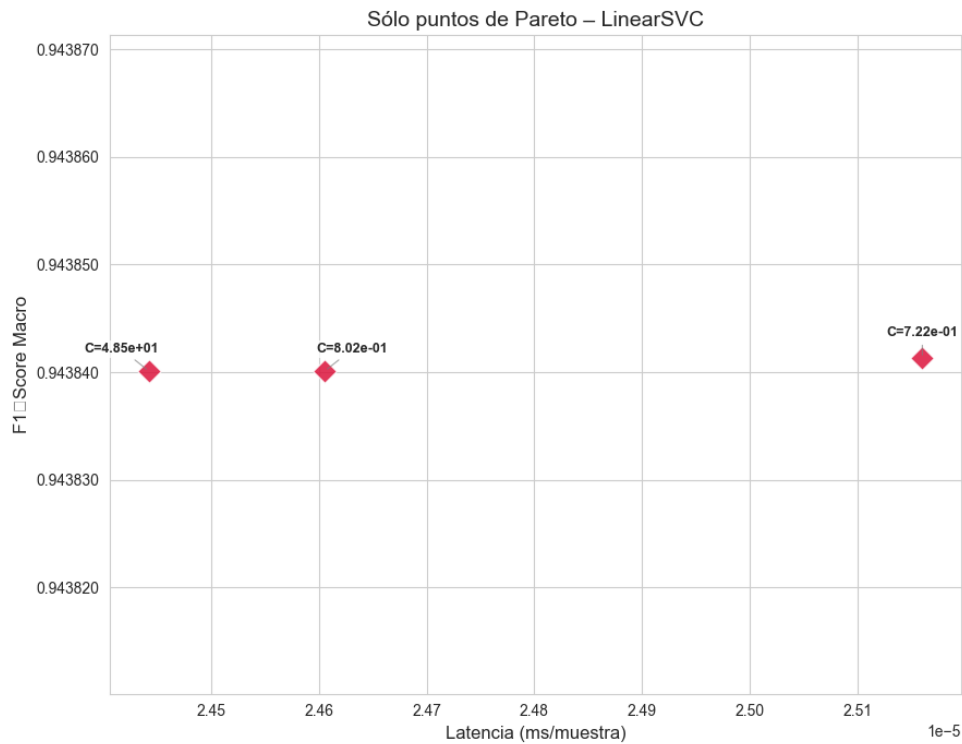


Figura 34. Frontera de Pareto - SVM (IOT-23)

Se han seleccionado los siguientes puntos de la frontera de Pareto para su evaluación en el set de test:

1. (48.498258)
2. (0.801714)
3. (0.721883)

Los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto son los siguientes:

Perfil	C	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	48.498258	0.943691	0.945963	0.000027
Candidato 2	0.801714	0.943695	0.945968	0.000025
Candidato 3	0.721883	0.943691	0.945963	0.000023

Tabla 31. Comparativa final de modelos SVM en el set de test (IOT-23)

En este caso, el **Candidato 2** es el modelo más eficiente, con un F1-Score de 0.943695 y una latencia de inferencia de 0.000025 ms, lo que lo posiciona como un modelo muy bueno para este dataset de IoT, aunque su rendimiento es significativamente inferior al de los modelos basados en árboles.

0.4.3.6 Multilayer Perceptron (MLP)

La gráfica de la frontera de Pareto obtenida para MLP es la siguiente:

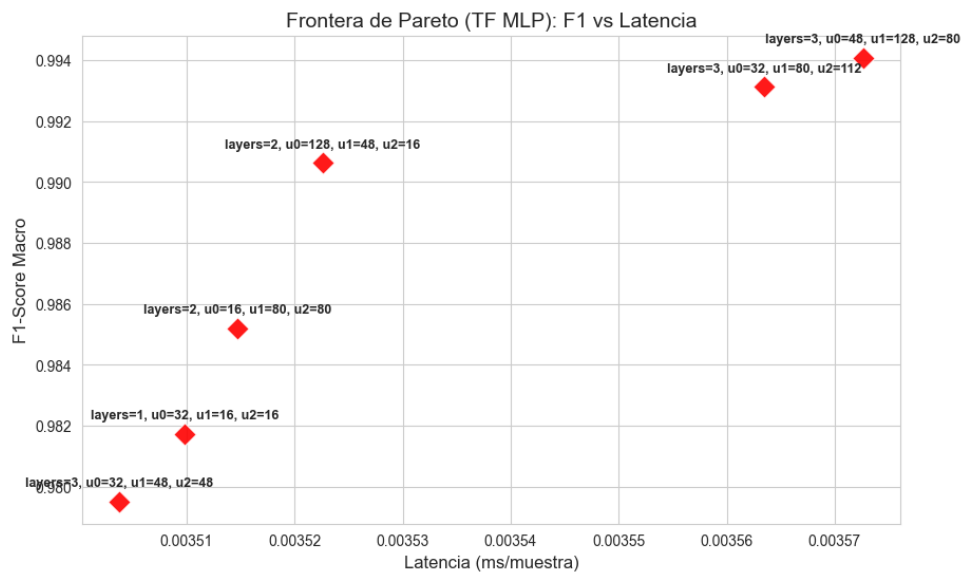


Figura 35. Frontera de Pareto - MLP (IOT-23)

La combinación de hiperparámetros seleccionada para su evaluación en el set de test es la siguiente:

1. (3, 32, 48, 48)
2. (1, 32, 0, 0)

3. (2, 16, 80, 0)
4. (2, 128, 48, 0)
5. (3, 32, 80, 112)
6. (3, 48, 128, 80)

Los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto son los siguientes:

Perfil	n_layers	n_units_l0	n_units_l1	n_units_l2	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	3	32	48	48	0.994819	0.995004	0.000717
Candidato 2	1	32	0	0	0.985158	0.985741	0.000621
Candidato 3	2	16	80	0	0.990610	0.990969	0.000719
Candidato 4	2	128	48	0	0.993502	0.993737	0.000772
Candidato 5	3	32	80	112	0.995042	0.995223	0.000919
Candidato 6	3	48	128	80	0.996544	0.996665	0.000906

Tabla 32. Comparativa final de modelos MLP en el set de test (IOT-23)

En este caso, nos quedamos con el **Candidato 1**, que ofrece un excelente rendimiento (F1-Score de 0.994819) y una latencia de inferencia de 0.000717 ms, lo que lo posiciona como un modelo muy bueno para este dataset de IoT, aunque su rendimiento sigue siendo inferior al de los modelos basados en árboles.

Bibliografía

- [1] Ayesha Alharthi, Meera Alaryani, and Sanaa Kaddoura. "A Comparative Study of Machine Learning and Deep Learning Models in Binary and Multiclass Classification for Intrusion Detection Systems". In: *Array* 26 (2025), p. 100406. DOI: [10.1016/j.array.2025.100406](https://doi.org/10.1016/j.array.2025.100406).
- [2] Fatima Jobran ALzaher and Asma AlJarullah. "Intrusion Detection Using Machine Learning and Deep Learning". In: *International Journal of Advanced Computer Science and Applications* 16.8 (2025), pp. 440–454.
- [3] Bouchra Hafid, Abdellatif Ezzouhairi, and Khalid Haddouch. "Strengthening Security in the Internet of Things (IoT): Integrated Approach of Intrusion Detection Systems (IDS) and Edge Computing". In: *2024 3rd International Conference on Embedded Systems and Artificial Intelligence (ESAI)*. 2024, pp. 1–9. DOI: [10.1109/ESAI62891.2024.10913549](https://doi.org/10.1109/ESAI62891.2024.10913549).
- [4] Md. Kamruzzaman et al. "Lightweight AI-Driven intrusion detection for SMEs: Comparative analysis of machine learning models on Ton_lot". In: *Intelligent Decision Technologies* (2025). URL: <https://api.semanticscholar.org/CorpusID:283327148>.
- [5] Sydney Mambwe Kasongo. "A Deep Learning Technique for Intrusion Detection System Using a Recurrent Neural Networks Based Framework". In: *Computer Communications* 196 (2023), pp. 57–73. DOI: [10.1016/j.comcom.2022.10.015](https://doi.org/10.1016/j.comcom.2022.10.015).
- [6] Vyshnavi Manduru et al. "A Light Gradient Boosted Model for Network Intrusion Detection". In: *2024 International Conference on Smart Systems for Electrical, Electronics, Communication and Computer Engineering (ICSSECC)* (2024), pp. 1–6. URL: <https://api.semanticscholar.org/CorpusID:272372361>.
- [7] Abdulrahman Mohammed, Muhanad Arif, and Ali Ali. "A multilayer perceptron artificial neural network approach for improving the accuracy of intrusion detection systems". In: *IAES International Journal of Artificial Intelligence (IJ-AI)* 9 (Dec. 2020), p. 609. DOI: [10.11591/ijai.v9.i4.pp609-615](https://doi.org/10.11591/ijai.v9.i4.pp609-615).
- [8] Nishank Parekh et al. "Network Intrusion Detection System Using Optuna". In: *2024 International Conference on IoT Based Control Networks and Intelligent Systems (ICICNIS)*. 2024, pp. 312–318. DOI: [10.1109/ICICNIS64247.2024.10823298](https://doi.org/10.1109/ICICNIS64247.2024.10823298).
- [9] Anshika Sharma and Himanshi Babbar. "Understanding IoT-23 Dataset: A Benchmark for IoT Security Analysis". In: *2024 4th International Conference on Intelligent Technologies (CONIT)*. 2024, pp. 1–5. DOI: [10.1109/CONIT61985.2024.10627334](https://doi.org/10.1109/CONIT61985.2024.10627334).

- [10] Gurbakhsis Singh and Meenakshi Bansal. "An Analytical Review of Deep Learning Models and Datasets for Anomaly-Based Intrusion Detection Systems". In: *Cuestiones de Fisioterapia* 52.3 (2023). DOI: [10.48047/qdgj7n28](https://doi.org/10.48047/qdgj7n28). URL: <https://doi.org/10.48047/qdgj7n28>.
- [11] Sudhanshu Sekhar Tripathy and Bichitrananda Behera. "Detection System: Advances and Challenges". In: *arXiv preprint arXiv:2506.02438* (2025). DOI: [10.48550/arXiv.2506.02438](https://doi.org/10.48550/arXiv.2506.02438). URL: <https://doi.org/10.48550/arXiv.2506.02438>.



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

ETS. de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga